JEST & VITEST

CHEAT SHEETS

Unit and Component Testing for JavaScript — Vitest First, Jest Alongside

Config · Watch Mode · describe / it · Hooks · test.each · Matchers · Asymmetric Matchers · Async · vi.fn · Spies · Module Mocks · Fake Timers · MSW · Snapshots · jsdom · Testing Library · Browser Mode · Coverage · Projects · CI · Jest Config · Migration · React Native

27 REFERENCE SHEETS

CONTENTS

INTRODUCTION & SETUP

01 Introduction: Vitest & Jest
02 Installation & Config
03 Running Tests & Watch Mode

WRITING TESTS

04 describe, it & test
05 Hooks, Setup Files & Test Context
06 Parameterized Tests & Modifiers

MATCHERS

07 Equality & Truthiness
08 Numbers, Strings & Collections
09 Asymmetric, Soft & Custom Matchers
10 Errors & Async Code

MOCKING

11 Mock Functions
12 Spies & Resetting Mocks
13 Module Mocking
14 Fake Timers & Dates
15 Mocking Network, Globals & Env

SNAPSHOTS & THE DOM

16 Snapshot Testing
17 DOM Environments
18 Testing Library
19 Vitest Browser Mode

COVERAGE, PERFORMANCE & CI

20 Coverage
21 Projects & Performance
22 Reporters & CI

JEST-SPECIFIC

23 Jest Setup & Config
24 Migrating Jest → Vitest
25 Jest in React Native

QUICK REFERENCE

26 Quick Reference — Matchers & Mocks
27 Quick Reference — CLI & Config

JV

Table of Contents

27 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 03

- 01 Introduction: Vitest & …
- 02 Installation & Config
- 03 Running Tests & Watch…

WRITING TESTS 04 – 06

- 04 describe, it & test
- 05 Hooks, Setup Files & T…
- 06 Parameterized Tests & …

MATCHERS 07 – 10

- 07 Equality & Truthiness
- 08 Numbers, Strings & Co…
- 09 Asymmetric, Soft & Cu…
- 10 Errors & Async Code

MOCKING 11 – 15

- 11 Mock Functions
- 12 Spies & Resetting Moc…
- 13 Module Mocking
- 14 Fake Timers & Dates
- 15 Mocking Network, Glo…

SNAPSHOTS & THE DOM 16 – 19

- 16 Snapshot Testing
- 17 DOM Environments
- 18 Testing Library
- 19 Vitest Browser Mode

COVERAGE, PERFORMANCE & CI 20 – 22

- 20 Coverage
- 21 Projects & Performance
- 22 Reporters & CI

JEST-SPECIFIC 23 – 25

- 23 Jest Setup & Config
- 24 Migrating Jest → Vitest
- 25 Jest in React Native

QUICK REFERENCE 26 – 27

- 26 Quick Reference — Ma…
- 27 Quick Reference — CLI…

Sheet 01

Introduction: Vitest & Jest

what a test runner does · Vitest vs Jest · Jest-compatible API · when to pick each · `npm i -D vitest` · first test

TWO RUNNERS, ONE API

A **test runner** finds test files, runs them, and reports which assertions failed. **Jest** (Meta, now OpenJS) made `describe` / `it` / `expect` the standard API for JavaScript. **Vitest** (from the Vite team) uses the same API but runs on Vite's transform pipeline, so TypeScript, ESM and JSX work with no extra config and watch mode is fast. This set teaches Vitest first and shows Jest beside it.

NOTES

- Pick **Vitest** for Vite, Vue, Svelte, Astro, Nuxt and most new projects
- Pick **Jest** for React Native and existing Jest suites
- Both run in Node by default; a DOM environment is opt-in (sheet 17)
- End-to-end browser testing is the Playwright set's job

UNDER THE HOOD

- Vitest transforms each file with Vite's plugins (TS, JSX, aliases) on demand, so it shares config with your app
- Jest uses its own module registry and transformers (Babel, ts-jest, SWC) configured separately

FIRST TEST

```
// src/cart.js
export function total(prices, discount = 0) {
  if (discount < 0 || discount >= 1) throw new RangeError('discount 0..1')
  return Math.round(prices.reduce((a, b) => a + b, 0) * (1 - discount) * 100) / 100
}

// src/cart.test.js
import { describe, it, expect } from 'vitest'
import { total } from './cart.js'

describe('total', () => {
  it('adds prices', () => {
    expect(total([2.5, 1.25])).toBe(3.75)
  })
  it('applies a discount', () => {
    expect(total([10, 10], 0.25)).toBe(15)
  })
})

// $ npm i -D vitest
// $ npx vitest          → watch mode
//  ✓ src/cart.test.js (2 tests) 3ms
```

CODE EXPLAINED

- → Test files end in `.test.js` / `.spec.js` (or `.ts`, `.tsx`) and sit next to the code
- → `describe` groups tests; `it` (or `test`) is one test; `expect(x).toBe(y)` asserts
- → Importing `describe`, `it`, `expect` from `vitest` keeps tests explicit (sheet 02 shows globals)
- → `npx vitest` watches by default in a terminal; `vitest run` runs once

JEST ↔ VITEST

- ⇆ `npm i -D jest` · run with `npx jest`
- ⇆ Jest provides `describe` / `it` / `expect` as globals, no import needed
- ⇆ TypeScript and ESM need a transformer in Jest (sheet 23)

TYPESCRIPT

- TS Name test files `*.test.ts`; Vitest runs TS directly, no build step
- TS Vitest only strips types; run `tsc --noEmit` separately to type-check

PREFER / AVOID

- ⇄ `console.log` checks by eye → `expect` assertions
- ⇄ one giant test → small tests named after behavior
- ⇄ tests in a separate tree far from the code → `*.test.ts` next to the module

VITEST VS JEST

AREA	VITEST	JEST
TS / ESM / JSX	Built in (Vite)	Needs a transformer
Config	vite.config / vitest.config	jest.config
Globals	Opt-in (globals: true)	On by default
Mocks	vi.*	jest.*
Watch mode	Default in terminal	--watch
Browser mode	Yes (real browser)	No (jsdom only)

TEST FILE PATTERNS

PATTERN	FOUND BY
*.test.{js,ts,jsx,tsx}	Both
*.spec.{js,ts}	Both
__tests__/*.js	Jest (Vitest via include)
*.test-d.ts	Vitest type tests

⚠ COMMON MISTAKES

- Mixing Jest and Vitest globals in one project and getting "vi is not defined" or "jest is not defined"
- Expecting Vitest to type-check; type errors only show with `tsc`
- Writing browser end-to-end flows as unit tests; use Playwright for those

✓ TIPS & BEST PRACTICES

- Keep tests next to the code (`cart.ts` + `cart.test.ts`)
- Add `"test": "vitest"` and `"test:ci": "vitest run"` scripts
- Run `tsc --noEmit` as a separate CI step

QUICK SUMMARY

- Vitest — Jest API on Vite
- Jest — the original; React Native default
- `describe` / `it` / `expect` — same in both
- `vitest` watches · `vitest run` once

Installation & Config

CONFIGURE ONCE, SHARED WITH VITE

Vitest reads the **test** block of your `vite.config.ts`, or a separate `vitest.config.ts` that wins if both exist. It inherits Vite's plugins and aliases, so tests resolve imports the same way the app does. The common settings are `environment` (node, jsdom, happy-dom), `globals`, `include`, `setupFiles` and `coverage`.

NOTES

- Install: `npm i -D vitest` (+ `jsdom` or `happy-dom` for DOM tests)
- Use `defineConfig` from `vitest/config` for typed options
- `setupFiles` run before **each** test file
- The Vite set covers `vite.config` itself

VITEST.CONFIG.TS

```ts
import { defineConfig } from 'vitest/config'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  resolve: { alias: { '@': new URL('./src', import.meta.url).pathname } },
  test: {
    environment: 'jsdom',            // 'node' (default) | 'happy-dom'
    globals: true,                   // describe / it / expect without imports
    include: ['src/**/*.{test,spec}.{ts,tsx}'],
    exclude: ['**/node_modules/**', 'e2e/**'],
    setupFiles: ['./vitest.setup.ts'],
    restoreMocks: true,              // undo spies after each test
    testTimeout: 5000,
  },
})

// package.json
// "scripts": { "test": "vitest", "test:ci": "vitest run --coverage" }
```

CODE EXPLAINED

→ `plugins` and `resolve.alias` are the same Vite options the app uses

→ `include` limits which files are tests; keep end-to-end specs out

→ `setupFiles` is where `@testing-library/jest-dom/vitest` and MSW start (sheets 15, 18)

→ `restoreMocks: true` saves writing cleanup in every file

JEST ↔ VITEST

⇆ Jest: `jest.config.ts` with `testEnvironment`, `testMatch`, `setupFilesAfterEnv` (sheet 23)

⇆ `globals: true` is Jest's default behavior

⇆ Jest doesn't read `vite.config`; aliases go in `moduleNameMapper`

TYPESCRIPT

TS With `globals: true`, add `"types": ["vitest/globals"]` to `tsconfig.json`

TS Name the file `vitest.config.ts`; it's typed through `defineConfig`

PREFER / AVOID

⇄ duplicating aliases in a test config → let Vitest reuse `vite.config`

⇄ per-file `afterEach(() => vi.restoreAllMocks())` → `restoreMocks: true`

⇄ one huge setup file → only global setup there; local setup in hooks

KEY TEST OPTIONS

OPTION	SETS
`environment`	node · jsdom · happy-dom · edge-runtime
`globals`	Global test APIs
`include / exclude`	Test file globs
`setupFiles`	Run before each file
`globalSetup`	Run once before everything
`testTimeout`	Per-test timeout (ms)
`restoreMocks / clearMocks`	Auto mock cleanup
`coverage`	Coverage options (sheet 20)

CONFIG LOOKUP

FILE	USED WHEN
`vitest.config.ts`	Wins if present
`vite.config.ts → test`	Otherwise
`--config path`	Explicit file
`vitest.workspace.ts`	Removed; use test.projects (sheet 21)

⚠ COMMON MISTAKES

- Having both `vite.config.ts` and `vitest.config.ts` and editing the ignored one
- Turning on `globals` without updating `tsconfig` types, so the editor shows errors
- Putting app setup (real servers, timers) in `setupFiles` that every test file pays for

✓ TIPS & BEST PRACTICES

- Start with the `test` block in `vite.config.ts`; split it out only when it grows
- Add `environment` per file (sheet 17) instead of making every test use jsdom
- Keep `include` tight so Vitest never picks up Playwright specs

QUICK SUMMARY

- `test` in `vite.config` or `vitest.config.ts`
- `environment` — node / jsdom / happy-dom
- `globals` — skip imports
- `setupFiles` — per-file setup

Sheet 03

Running Tests & Watch Mode

vitest vs vitest run · filters by name / file · -t · --last-failed · --changed · --ui · --project

RUN ONLY WHAT CHANGED

`vitest` starts **watch mode**: it runs the suite, then reruns only the tests affected by each file you save. `vitest run` runs once and exits, which is what CI uses. Filter by file path (any part of the name), by test name with `-t`, by failed tests with `--last-failed`, or by Git changes with `--changed`.

NOTES

- In watch mode press `a` all, `f` failed, `p` filter file, `t` filter name, `q` quit
- Vitest switches to run-once automatically in CI (`CI` env var)
- `--ui` opens a browser dashboard (`@vitest/ui`)
- Add `.only` temporarily to focus one test (sheet 06)

UNDER THE HOOD

- Vitest keeps a module graph of every test file and its imports
- On save it reruns only test files whose graph includes the changed file
- `--changed` asks Git which files changed and uses the same graph

COMMAND LINE

```
npx vitest                          # watch mode
npx vitest run                      # run once
npx vitest cart                     # files whose path contains "cart"
npx vitest src/cart.test.ts:12      # the test on line 12
npx vitest -t "applies a discount"  # by test name (regex)
npx vitest run --last-failed        # only last run's failures
npx vitest run --changed            # tests affected by uncommitted changes
npx vitest run --changed origin/main
npx vitest --ui                     # browser dashboard
npx vitest --project unit           # one project (sheet 21)
npx vitest run --bail 1             # stop after the first failure
npx vitest related src/cart.ts      # tests that import this file
```

CODE EXPLAINED

→ Path filters are substrings, so `cart` matches `src/cart.test.ts` and `cartItem.test.ts`

→ `file:line` runs one test by its line number

→ `--changed origin/main` is handy in PR builds

→ `vitest related` is for lint-staged / pre-commit hooks

JEST ↔ VITEST

⇆ Jest: `npx jest` runs once; `--watch` (changed since commit) or `--watchAll`

⇆ `-t` is the same; `--onlyChanged` / `-o` and `--findRelatedTests` match `--changed` / `related`

⇆ Jest's last-failed: press `f` in watch mode, or `--onlyFailures`

TYPESCRIPT

TS Vitest runs `.ts` tests directly; no `tsc` build before running

TS Use `vitest --typecheck` to also run `*.test-d.ts` type tests

PREFER / AVOID

⇄ `vitest run` while editing → `vitest` watch mode

⇄ rerunning everything after one failure → `f` in watch or `--last-failed`

⇄ leaving `.only` in commits → `-t` filters on the command line

WATCH MODE KEYS

KEY	DOES
a	Rerun all
f	Rerun failed
u	Update snapshots
p	Filter by file name
t	Filter by test name
h	Help
q	Quit

USEFUL FLAGS

FLAG	DOES
run	Run once
-t PATTERN	Test name filter
--last-failed	Previous failures
--changed [ref]	Git-affected tests
--reporter X	Output style (sheet 22)
--coverage	Coverage (sheet 20)
--ui	Dashboard

⚠ COMMON MISTAKES

- Running `vitest` in a CI script without `run` on a system where `CI` isn't set; it waits forever
- Filtering with `-t` but a path filter that excludes the file
- Forgetting a test is skipped by `.only` elsewhere in the file

✓ TIPS & BEST PRACTICES

- Keep `vitest` running in a terminal split while you code
- Use `vitest related` in pre-commit to test only touched files
- Use `--ui` to browse failures and the module graph on big suites

QUICK SUMMARY

- `vitest` — watch · `vitest run` — once
- path filter / `-t` — select tests
- `--last-failed` / `--changed` — rerun smart
- `--ui` — dashboard

describe, it & test

STRUCTURE THAT READS LIKE A SPEC

`describe(name, fn)` groups related tests and can nest; `it(name, fn)` and `test(name, fn)` are the same function and define one test. Names combine into a sentence in the report: `total > applies a discount`. Tests run in the order written within a file; files run in parallel workers.

NOTES

- A test passes if its function returns (or its promise resolves) without throwing
- An `async` test can `await` anything (sheet 10)
- Third argument: timeout in ms or options `{ timeout, retry }`
- Empty `describe` blocks are allowed but report nothing

GROUPING TESTS

```
import { describe, it, test, expect } from 'vitest'
import { Cart } from './cart'

describe('Cart', () => {
  describe('add', () => {
    it('adds a new item', () => {
      const cart = new Cart()
      cart.add('apple', 2)
      expect(cart.count('apple')).toBe(2)
    })

    it('increases the quantity of an existing item', () => {
      const cart = new Cart()
      cart.add('apple', 2)
      cart.add('apple', 1)
      expect(cart.count('apple')).toBe(3)
    })
  })

  test('total is 0 when empty', () => {
    expect(new Cart().total()).toBe(0)
  })

  it('loads remote prices', async () => {
    await expect(new Cart().loadPrices()).resolves.toBeUndefined()
  }, 10_000)                              // per-test timeout
})
```

CODE EXPLAINED

- → Report names join with `>`: `Cart > add > adds a new item`
- → Each test builds its own `Cart`, so tests stay independent
- → `it` and `test` are aliases; pick one style per project
- → The last argument sets a timeout for slow tests

JEST ↔ VITEST

- ⇆ Identical in Jest: `describe`, `it`, `test` (as globals)
- ⇆ Jest timeout: same third argument, or `jest.setTimeout(ms)`
- ⇆ Vitest also accepts an options object: `it(name, { retry: 2 }, fn)`

TYPESCRIPT

- TS Annotate test data with `satisfies` to catch shape mistakes: `const item = {…} satisfies Item`
- TS Files can be `.test.ts` or `.test.tsx`; both run directly

PREFER / AVOID

- ⇄ `it("works")` → names describing behavior: `it("rejects a negative discount")`
- ⇄ shared `let cart` mutated by every test → build it in `beforeEach` or each test
- ⇄ deep nesting of `describe` → two levels is usually enough

TEST API

FUNCTION	PURPOSE
`describe(name, fn)`	Group tests
`it / test(name, fn, timeout?)`	One test
`describe.each / test.each`	Parameterized (sheet 06)
`.only / .skip / .todo`	Focus / skip / placeholder
`expect(value)`	Start an assertion

NAMING STYLES

STYLE	EXAMPLE
`Behavior`	it("rejects a negative discount")
`Given / when / then`	it("given an empty cart, total is 0")
`Method + case`	describe("add") → it("increases quantity")

⚠ COMMON MISTAKES

- Putting setup code directly in `describe` bodies; it runs at collection time, before any hooks
- Tests that depend on the previous test's side effects
- Async tests without `await` or `return`, which pass before the work finishes

✓ TIPS & BEST PRACTICES

- Read the report as a spec; if the names don't make sense together, rename them
- Keep one behavior per test
- Use `test.todo("…")` to list cases you still need to write

QUICK SUMMARY

- `describe` — group
- `it` / `test` — one test
- names join into a sentence
- third arg — timeout or options

Sheet 05

Hooks, Setup Files & Test Context

beforeEach / afterEach · beforeAll / afterAll · cleanup return functions · setupFiles · globalSetup · test.extend fixtures

SETUP AND TEARDOWN AROUND TESTS

Hooks run code around tests in their `describe` block: `beforeEach` / `afterEach` around every test, `beforeAll` / `afterAll` once for the block. A hook can **return a cleanup function**. Project-wide setup goes in **`setupFiles`** (each test file) or **`globalSetup`** (once per run). Vitest also has **`test.extend`** fixtures, pytest-style setup that only runs for tests that use it.

NOTES

- Hooks in an outer `describe` run before inner ones
- Async hooks can `await`
- `globalSetup` runs in a separate context; pass data with `provide` / `inject`
- Each test gets a context object as its first argument

UNDER THE HOOD

- Each test file runs in its own isolated module graph (by default), so module state never leaks between files
- `setupFiles` run inside that isolated context before every file; `globalSetup` runs once in the main process
- `test.extend` fixtures are created lazily, only when a test destructures them

HOOKS AND FIXTURES

```ts
import { beforeEach, afterEach, beforeAll, test as base, expect } from 'vitest'

let server: TestServer
beforeAll(async () => {
  server = await startServer()
  return () => server.close()          // cleanup when the block ends
})

beforeEach(() => {
  localStorage.clear()
})

afterEach(() => {
  vi.useRealTimers()
})

// test.extend: fixtures only built when requested
const test = base.extend<{ db: Db; user: User }>({
  db: async ({}, use) => {
    const db = await openTestDb()
    await use(db)                      // test runs here
    await db.close()
  },
  user: async ({ db }, use) => {
    await use(await db.createUser('ann'))
  },
})

test('user has no orders', async ({ user, db }) => {
  expect(await db.orders(user.id)).toEqual([])
})
```

CODE EXPLAINED

- → A function returned from `beforeAll` / `beforeEach` runs as its teardown
- → In `test.extend`, code after `await use(…)` is teardown
- → `user` depends on `db`; Vitest builds them in order
- → Tests that don't mention `db` never open the database

JEST ↔ VITEST

- ⇆ Jest hooks are the same; returning a cleanup function is Vitest-only
- ⇆ No `test.extend` in Jest; use hooks or helper functions
- ⇆ Jest: `setupFiles` (before framework) vs `setupFilesAfterEnv` (after; where matchers go)

TYPESCRIPT

- TS Type fixtures with the generic: `base.extend<{ db: Db }>({ … })`
- TS Context is typed too: `test("x", ({ task, expect }) => …)`

PREFER / AVOID

- ⇄ module-level `let` + `beforeEach` for everything → `test.extend` fixtures
- ⇄ expensive `beforeEach` → `beforeAll` + reset state per test
- ⇄ teardown in a separate `afterAll` far from setup → return a cleanup function

HOOKS

HOOK	RUNS
`beforeAll(fn)`	Once before the block
`beforeEach(fn)`	Before every test in the block
`afterEach(fn)`	After every test
`afterAll(fn)`	Once after the block
`return () => …`	Cleanup (Vitest)
`onTestFinished(fn)`	Cleanup registered inside a test (Vitest)

SETUP LEVELS

LEVEL	WHERE	RUNS
`Test`	beforeEach / test.extend	Every test
`Block`	beforeAll in describe	Once per describe
`File`	setupFiles	Once per test file
`Run`	globalSetup	Once per run

⚠ COMMON MISTAKES

- Using `beforeAll` for state tests modify, so test order changes results
- Expecting variables set in `globalSetup` to be visible in tests; use `provide` / `inject`
- Forgetting to `await` async work in hooks

✓ TIPS & BEST PRACTICES

- Return cleanup from hooks to keep setup and teardown together
- Use `test.extend` for resources only some tests need
- Put matchers and global mocks (MSW) in `setupFiles`

QUICK SUMMARY

- `beforeEach` / `afterEach` — around every test
- `beforeAll` / `afterAll` — once per block
- return a function — cleanup
- `test.extend` — lazy fixtures

Sheet 06

Parameterized Tests & Modifiers

test.each · test.for · .only / .skip / .todo · .fails · .concurrent · skipIf / runIf · tags

CASES IN A TABLE, TESTS WITH SWITCHES

test.each (and describe.each) runs one test per row of data, with the row values in the test name. Vitest's **test.for** is similar but passes each case as one argument and gives you the test context. **Modifiers** change how a test runs: **.only** focuses it, **.skip** skips it, **.todo** lists it as unwritten, **.fails** expects it to fail, and **.concurrent** runs it in parallel with siblings.

NOTES

- %s, %d, %o format values into test names; $name reads object fields
- skipIf(cond) and runIf(cond) decide at collection time
- .only in CI fails the run when allowOnly is false (the default in CI)
- Vitest 4.1+ adds test tags for grouping and filtering

UNDER THE HOOD

- .concurrent tests in one file share a worker and run their async parts interleaved
- Shared module state or fake timers break under .concurrent
- Use the context's expect (({ expect }) => …) so snapshot and assertion counts stay per test

EACH, FOR AND MODIFIERS

```
test.each([
  [[], 0, 0],
  [[10, 10], 0.25, 15],
  [[9.99], 0, 9.99],
])('total(%o, %d) → %d', (prices, discount, expected) => {
  expect(total(prices, discount)).toBe(expected)
})

test.each([
  { country: 'US', rate: 0.07 },
  { country: 'DE', rate: 0.19 },
])('tax for $country is $rate', ({ country, rate }) => {
  expect(taxRate(country)).toBe(rate)
})

test.for(['USD', 'EUR'])('formats %s', (cur, { expect }) => {
  expect(format(1, cur)).toMatch(/1/)
})

test.skipIf(process.platform === 'win32')('uses symlinks', () => {})
test.todo('handles refunds over 90 days')
test.fails('known bug #142', () => { expect(round(0.1 + 0.2)).toBe(0.3) })
test.concurrent('fetches a', async ({ expect }) => { /* … */ })
```

CODE EXPLAINED

- → Array rows spread into arguments; object rows arrive as one argument you destructure
- → $country in the name pulls the field from each object row
- → test.for passes each case whole and the context second, so expect is per test
- → .fails passes only while the assertion fails, like pytest's strict xfail

JEST ↔ VITEST

- ⇆ Jest has test.each and describe.each (arrays, objects, or a tagged-template table)
- ⇆ No test.for, skipIf, runIf or .fails in Jest; use .failing and plain if
- ⇆ .concurrent exists in Jest but is less used

TYPESCRIPT

- TS test.each infers row types; add as const for literal types
- TS Type object rows: test.each<{ country: string; rate: number }>([…])

PREFER / AVOID

- ⇄ a for loop around expect → test.each
- ⇄ committing .only → -t on the command line
- ⇄ .skip with no note → .todo or a skip with an issue link in the name

MODIFIERS

MODIFIER	EFFECT
.only	Run only these in the file
.skip	Skip
.todo	Placeholder, reported as todo
.fails	Pass only if it fails (Vitest)
.concurrent	Run in parallel with siblings
.skipIf(c) / .runIf(c)	Conditional (Vitest)
.sequential	Force order inside concurrent

NAME PLACEHOLDERS

TOKEN	INSERTS
%s	String
%d / %i	Number
%o	Object (pretty)
%#	Row index
$field	Object row field
%%	Literal %

⚠ COMMON MISTAKES

- Mutating a shared row object in one case and breaking the next
- Leaving .only in a file; CI fails (or locally, other tests silently don't run)
- Using .concurrent with fake timers or shared mocks

✓ TIPS & BEST PRACTICES

- Use object rows with $field names for readable reports
- Put the expected value in each row so the table is the spec
- Use test.todo to sketch cases before writing them

QUICK SUMMARY

- test.each — table-driven tests
- test.for — cases + context (Vitest)
- .only / .skip / .todo / .fails
- .concurrent — parallel within a file

Sheet 07

Equality & Truthiness

toBe · toEqual · toStrictEqual · toBeNull / toBeUndefined / toBeDefined · toBeTruthy / toBeFalsy · .not

SAME OBJECT, SAME SHAPE, OR SAME EXACTLY

`toBe` compares with `Object.is`: right for numbers, strings, booleans and checking the **same** object. `toEqual` compares **structure** recursively, ignoring `undefined` properties. `toStrictEqual` also checks `undefined` keys, array holes and class types. `.not` inverts any matcher.

NOTES

- `toBe(0.1 + 0.2)` fails; floats need `toBeCloseTo` (sheet 08)
- `toEqual` treats `{ a: undefined }` and `{}` as equal
- `toStrictEqual` fails if classes differ
- Failure output shows a diff of the two values

PICKING THE RIGHT EQUALITY

```
expect(2 + 2).toBe(4)
expect('ok').toBe('ok')
expect(cart).toBe(cart)                       // same reference

expect({ id: 1, tags: ['a'] }).toEqual({ id: 1, tags: ['a'] })   // ✓
expect({ id: 1, tags: ['a'] }).toBe({ id: 1, tags: ['a'] })      // ✗ different objects

expect({ id: 1, note: undefined }).toEqual({ id: 1 })          // ✓
expect({ id: 1, note: undefined }).toStrictEqual({ id: 1 })    // ✗

class Money { constructor(public cents: number) {} }
expect(new Money(5)).toEqual({ cents: 5 })        // ✓
expect(new Money(5)).toStrictEqual({ cents: 5 })  // ✗ class differs

expect(findUser(99)).toBeNull()
expect(config.port).toBeDefined()
expect(order.paidAt).toBeUndefined()
expect(cart.items.length).toBeTruthy()
expect(errors).not.toContain('timeout')
expect(NaN).toBeNaN()
```

CODE EXPLAINED

- → Use `toBe` for primitives and identity; use `toEqual` for objects and arrays
- → `toStrictEqual` catches accidental `undefined` fields in API responses
- → `toBeNull` / `toBeUndefined` give clearer failures than `toBe(null)`
- → `toBeTruthy` accepts any truthy value; be specific when you can

JEST ↔ VITEST

- ⇆ All of these are identical in Jest
- ⇆ Same `Object.is` semantics for `toBe` in both

TYPESCRIPT

- TS Matchers are typed loosely; `expect(x).toEqual<User>(…)` is not checked
- TS Use `expectTypeOf(x).toEqualTypeOf<User>()` in `*.test-d.ts` files for type-level checks

PREFER / AVOID

- ⇄ `expect(a === b).toBe(true)` → `expect(a).toBe(b)` (better failure output)
- ⇄ `toEqual` on API payloads → `toStrictEqual` when `undefined` fields matter
- ⇄ `toBeTruthy()` for a count → `toBeGreaterThan(0)` or `toHaveLength(n)`

EQUALITY MATCHERS

MATCHER	COMPARES
`toBe(v)`	Object.is (identity / primitives)
`toEqual(v)`	Deep structure, ignores undefined props
`toStrictEqual(v)`	Deep + undefined + class + holes
`toMatchObject(v)`	Subset of properties (sheet 08)

TRUTHINESS MATCHERS

MATCHER	PASSES FOR
`toBeNull()`	null
`toBeUndefined()`	undefined
`toBeDefined()`	Not undefined
`toBeTruthy()`	Any truthy value
`toBeFalsy()`	false, 0, "", null, undefined, NaN
`toBeNaN()`	NaN

⚠ COMMON MISTAKES

- `toBe` on objects and arrays, which compares references
- Assuming `toEqual` catches extra `undefined` fields
- Using `toBeTruthy` where a specific value matters and missing a wrong value

✓ TIPS & BEST PRACTICES

- Default to `toEqual` for data and `toBe` for primitives
- Use `toStrictEqual` in API contract tests
- Write the expected value as a literal so the diff reads well

QUICK SUMMARY

- `toBe` — identity / primitives
- `toEqual` — deep structure
- `toStrictEqual` — deep + types + undefined
- `.not` — invert

Numbers, Strings & Collections

MATCHERS FOR EVERYDAY VALUES

Number matchers compare sizes and floats within a tolerance. `toMatch` checks strings against a regex or substring. `toContain` checks an item in an array (by ===) or a substring; `toContainEqual` checks by structure. `toHaveLength`, `toHaveProperty` and `toMatchObject` check shape without spelling out the whole object.

NOTES

- `toBeCloseTo(v, digits)` — default 2 decimal digits
- `toHaveProperty("a.b[0]", v)` takes a path
- `toMatchObject` ignores extra properties
- Works on Sets and Maps too: `toContain` on a Set checks membership

NUMBERS, STRINGS, ARRAYS, OBJECTS

```
expect(0.1 + 0.2).toBeCloseTo(0.3)          // 2 digits by default
expect(price).toBeCloseTo(19.99, 2)
expect(items.length).toBeGreaterThan(0)
expect(age).toBeGreaterThanOrEqual(18)
expect(latency).toBeLessThan(200)

expect(message).toMatch(/order \d+ shipped/)
expect(message).toMatch('shipped')           // substring
expect(slug).not.toMatch(/\s/)

expect(['apple', 'pear']).toContain('pear')
expect([{ id: 1 }, { id: 2 }]).toContainEqual({ id: 2 })
expect(cart.items).toHaveLength(3)

expect(order).toHaveProperty('status', 'paid')
expect(order).toHaveProperty('address.city', 'Tripoli')
expect(order).toHaveProperty(['lines', 0, 'sku'])

expect(order).toMatchObject({
  status: 'paid',
  lines: [{ sku: 'A1' }],                    // partial match inside arrays too
})
```

CODE EXPLAINED

- → `toBeCloseTo(v, 2)` passes when the difference is below 0.005
- → `toContain` on objects compares references; use `toContainEqual` for objects
- → `toHaveProperty` with a path avoids chains of `?.` in assertions
- → `toMatchObject` checks only the properties you list

JEST ↔ VITEST

- ⇆ Identical in Jest
- ⇆ Both accept arrays of keys in `toHaveProperty`

TYPESCRIPT

- TS `toHaveProperty` paths aren't type-checked; a typo just fails at run time
- TS For typed partials, build the expected object with `Partial<Order>`

PREFER / AVOID

- ⇄ `expect(arr.includes(x)).toBe(true)` → `expect(arr).toContain(x)`
- ⇄ `toEqual` on a huge object to check two fields → `toMatchObject`
- ⇄ `toBe(0.3)` on float math → `toBeCloseTo(0.3)`

NUMBER MATCHERS

MATCHER	PASSES WHEN
`toBeGreaterThan(n)`	> n
`toBeGreaterThanOrEqual(n)`	>= n
`toBeLessThan(n)`	< n
`toBeLessThanOrEqual(n)`	<= n
`toBeCloseTo(n, d)`	Within 10^-d / 2

STRING & COLLECTION MATCHERS

MATCHER	CHECKS
`toMatch(re | str)`	String matches
`toContain(x)`	Item (===) or substring
`toContainEqual(x)`	Item by structure
`toHaveLength(n)`	.length
`toHaveProperty(path, v?)`	Property exists / value
`toMatchObject(o)`	Subset of properties

⚠ COMMON MISTAKES

- `toContain({ id: 2 })` on an array of objects (always fails: different references)
- `toBeCloseTo(19.99)` on money where 2 digits isn't the tolerance you meant
- Using `toMatch` with an unescaped `.` or `?` in a regex

✓ TIPS & BEST PRACTICES

- Prefer `toMatchObject` for API responses with many incidental fields
- Pass the precision to `toBeCloseTo` explicitly
- Use `toHaveLength` for clearer failure messages than comparing `.length`

QUICK SUMMARY

- `toBeCloseTo` — floats
- `toMatch` — regex / substring
- `toContain` / `toContainEqual` — membership
- `toHaveProperty` / `toMatchObject` — shape

Sheet 09

Asymmetric, Soft & Custom Matchers

expect.any · expect.objectContaining · arrayContaining · stringMatching · expect.soft · expect.poll · expect.extend

MATCH "SOMETHING LIKE THIS"

Asymmetric matchers stand in for a value inside `toEqual`, `toHaveBeenCalledWith` and friends: `expect.any(Number)` matches any number, `expect.objectContaining({...})` any object with those fields. **`expect.soft`** records a failure and keeps going, **`expect.poll`** retries until a condition holds, and **`expect.extend`** adds your own matchers.

NOTES

- Use asymmetric matchers for IDs, dates and other values you can't predict
- `expect.not.objectContaining` inverts
- Soft failures still fail the test at the end
- Put `expect.extend` in a setup file so all tests get it

FLEXIBLE ASSERTIONS

```
expect(createOrder({ sku: 'A1' })).toEqual({
  id: expect.any(String),
  createdAt: expect.any(Date),
  sku: 'A1',
  lines: expect.arrayContaining([expect.objectContaining({ qty: 1 })]),
  status: expect.stringMatching(/^(new|pending)$/),
  meta: expect.anything(),                  // anything except null / undefined
})

expect(send).toHaveBeenCalledWith(expect.objectContaining({ to: 'ann@x.com' }))

expect.soft(res.status).toBe(200)           // keep going after a failure
expect.soft(res.headers['cache-control']).toBe('no-store')

await expect.poll(() => queue.size(), { timeout: 2000 }).toBe(0)

expect.extend({
  toBeValidSku(received: string) {
    const pass = /^[A-Z]\d{2,}$/.test(received)
    return { pass, message: () => `expected ${received} ${pass ? 'not ' : ''}to be a SKU` }
  },
})
expect('A12').toBeValidSku()
```

CODE EXPLAINED

- → `expect.any(Date)` matches any `Date` instance without fixing its value
- → `arrayContaining` passes if the listed items appear anywhere, in any order
- → `expect.poll` calls the function until the matcher passes or the timeout ends
- → A custom matcher returns `pass` and a `message` function

JEST ↔ VITEST

- ⇆ Jest has the same asymmetric matchers and `expect.extend`
- ⇆ No `expect.soft` or `expect.poll` in Jest

TYPESCRIPT

- TS Declare custom matchers: `interface Assertion<T> { toBeValidSku(): T }` in `declare module "vitest"`
- TS Jest: augment `jest.Matchers<R>` instead

PREFER / AVOID

- ⇄ deleting `id` / `createdAt` before comparing → `expect.any(...)`
- ⇄ `setTimeout` + assert for async state → `expect.poll`
- ⇄ the same complex check copied into many tests → `expect.extend`

ASYMMETRIC MATCHERS

MATCHER	MATCHES
`expect.any(Ctor)`	Instance of / typeof
`expect.anything()`	Not null / undefined
`expect.objectContaining(o)`	Object with these props
`expect.arrayContaining(a)`	Array with these items
`expect.stringContaining(s)`	Substring
`expect.stringMatching(re)`	Regex
`expect.closeTo(n, d)`	Float within tolerance

SPECIAL EXPECT FORMS

FORM	USE
`expect.soft(x)`	Record and continue (Vitest)
`expect.poll(fn, opts)`	Retry until true (Vitest)
`expect.assertions(n)`	Exactly n assertions ran
`expect.extend({…})`	Custom matchers

⚠ COMMON MISTAKES

- Using `expect.anything()` and accepting values you should check
- Writing a custom matcher without a useful failure message
- Polling with no timeout and hanging the suite

✓ TIPS & BEST PRACTICES

- Combine `toEqual` with asymmetric matchers to assert the whole shape at once
- Use `expect.soft` for several independent checks on one response
- Write custom matcher messages that say what was expected and what arrived

QUICK SUMMARY

- `expect.any` / `objectContaining` — flexible values
- `expect.soft` — continue after failure
- `expect.poll` — retry until true
- `expect.extend` — your own matchers

Errors & Async Code

TESTING THROWS AND PROMISES

To test that code **throws**, wrap the call in a function: `expect(() => fn()).toThrow(...)`. For **promises**, make the test `async` and `await` `expect(promise).resolves` or `.rejects`. Forgetting the `await` is the classic bug: the test finishes before the promise settles and passes no matter what.

NOTES

- `toThrow` accepts a string (substring), regex, class or error object
- Always `await` (or `return`) `.resolves` / `.rejects`
- `expect.assertions(n)` fails if fewer assertions ran, catching skipped `catch` blocks
- Unhandled rejections during a test fail the run

UNDER THE HOOD

- An `async` test returns a promise; the runner waits for it
- A `.rejects` chain without `await` creates a promise nobody waits for, so the test ends first
- Vitest reports such dangling promises as errors when they reject later

THROWS AND PROMISES

```
test('rejects a bad discount', () => {
  expect(() => total([10], 2)).toThrow(RangeError)
  expect(() => total([10], 2)).toThrow('discount 0..1')
  expect(() => total([10], 2)).toThrow(/0\.\.1/)
})

test('loads a user', async () => {
  await expect(fetchUser(1)).resolves.toMatchObject({ id: 1 })
})

test('fails for a missing user', async () => {
  await expect(fetchUser(999)).rejects.toThrow(NotFoundError)
})

test('plain await style', async () => {
  const user = await fetchUser(1)
  expect(user.name).toBe('Ann')
})

test('catch block runs', async () => {
  expect.assertions(1)
  try {
    await fetchUser(999)
  } catch (e) {
    expect(e).toBeInstanceOf(NotFoundError)
  }
})
```

CODE EXPLAINED

- → `expect(total(...))` without the arrow function throws before `expect` runs
- → `.rejects.toThrow(Class)` checks the rejection reason's type
- → Plain `await` + assertions is often the clearest async style
- → `expect.assertions(1)` fails the test if `fetchUser` unexpectedly resolves

JEST ↔ VITEST

- ⇆ Same in Jest, except Jest 30 removed the `toThrowError` alias; use `toThrow`
- ⇆ Jest still allows `done` callbacks; Vitest doesn't (use promises)

TYPESCRIPT

- TS In `catch (e)`, `e` is `unknown`; narrow with `toBeInstanceOf` or `if (e instanceof …)`
- TS `await expect(p).resolves` keeps `p`'s resolved type for chained matchers

PREFER / AVOID

- ⇄ `expect(p).rejects…` without `await` → `await expect(p).rejects…`
- ⇄ try/catch without `expect.assertions` → `.rejects` or add `expect.assertions(n)`
- ⇄ `done` callbacks → `async` / `await`

TOTHROW ARGUMENTS

ARGUMENT	CHECKS
`(none)`	Anything thrown
`"text"`	Message contains text
`/regex/`	Message matches
`ErrorClass`	instanceof
`new Error("x")`	Message equals "x"

ASYNC FORMS

FORM	USE
`await expect(p).resolves.X`	Assert resolved value
`await expect(p).rejects.X`	Assert rejection
`const v = await p`	Then normal asserts
`expect.assertions(n)`	Guard code paths
`expect.hasAssertions()`	At least one assertion ran

⚠ COMMON MISTAKES

- `expect(fn()).toThrow()` instead of `expect(() => fn()).toThrow()`
- Missing `await` on `.resolves` / `.rejects`
- Using `toThrowError` after upgrading to Jest 30

✓ TIPS & BEST PRACTICES

- Use `toThrow(ErrorClass)` plus a message regex for precise checks
- Turn on the `@typescript-eslint/no-floating-promises` lint rule to catch missing awaits
- Prefer `.rejects` over try/catch in tests

QUICK SUMMARY

- `expect(() => fn()).toThrow()` — sync errors
- `await expect(p).resolves / .rejects` — promises
- `async` tests — await anything
- `expect.assertions(n)` — guard paths

Mock Functions

FAKE FUNCTIONS THAT REMEMBER

`vi.fn()` creates a **mock function**: callable, returns `undefined` by default, and records every call's arguments, `this` and result. Configure what it returns with `mockReturnValue`, `mockResolvedValue` (promises) or `mockImplementation`, then assert how it was called with the `toHaveBeenCalled…` matchers.

NOTES

- `…Once` variants apply to the next call only, then fall back
- `vi.fn(impl)` sets the implementation up front
- `.mock.calls` is an array of argument arrays
- Pass mocks as callbacks or inject them as dependencies

VI.FN IN PRACTICE

```
import { vi, test, expect } from 'vitest'

test('calls onSave with the form data', () => {
  const onSave = vi.fn()
  submitForm({ name: 'Ann' }, onSave)
  expect(onSave).toHaveBeenCalledTimes(1)
  expect(onSave).toHaveBeenCalledWith({ name: 'Ann' })
})

test('retries on failure', async () => {
  const send = vi.fn()
    .mockRejectedValueOnce(new Error('timeout'))
    .mockResolvedValueOnce({ ok: true })
  await expect(sendWithRetry(send)).resolves.toEqual({ ok: true })
  expect(send).toHaveBeenCalledTimes(2)
})

test('custom implementation', () => {
  const price = vi.fn((sku: string) => (sku === 'A1' ? 5 : 0))
  expect(total(['A1', 'B2'], price)).toBe(5)
  expect(price.mock.calls).toEqual([['A1'], ['B2']])
  expect(price).toHaveReturnedWith(5)
  expect(price).toHaveBeenLastCalledWith('B2')
})
```

CODE EXPLAINED

- → `toHaveBeenCalledWith` passes if any call matched; use `…NthCalledWith(n, …)` for a specific call
- → Chained `…Once` calls define a sequence of results
- → `mock.calls` and `mock.results` let you inspect everything when matchers aren't enough
- → Injecting `send` as a parameter avoids module mocking entirely

JEST ↔ VITEST

- ⇆ Jest: `jest.fn()` with the same API
- ⇆ Jest 30 removed aliases like `toBeCalled`, `toBeCalledWith`, `lastCalledWith`; use the `toHaveBeen…` names

TYPESCRIPT

- TS Type a mock: `vi.fn<(sku: string) => number>()`
- TS Wrap an existing mock for types: `vi.mocked(fn).mockReturnValue(…)`

PREFER / AVOID

- ⇄ `mock.calls.length === 1` → `toHaveBeenCalledTimes(1)`
- ⇄ checking every internal call → assert on results; check calls only at boundaries
- ⇄ `mockImplementation(() => Promise.resolve(x))` → `mockResolvedValue(x)`

CONFIGURE

METHOD	EFFECT
`mockReturnValue(v)` / `…Once`	Return v
`mockResolvedValue(v)` / `…Once`	Return Promise.resolve(v)
`mockRejectedValue(e)` / `…Once`	Return Promise.reject(e)
`mockImplementation(fn)` / `…Once`	Run fn
`mockName("x")`	Name in failure output

CALL MATCHERS

MATCHER	CHECKS
`toHaveBeenCalled()`	At least once
`toHaveBeenCalledTimes(n)`	Exactly n
`toHaveBeenCalledWith(…)`	Some call had these args
`toHaveBeenLastCalledWith(…)`	Last call
`toHaveBeenNthCalledWith(n, …)`	nth call
`toHaveReturnedWith(v)`	Some call returned v

⚠ COMMON MISTAKES

- Asserting `toHaveBeenCalledWith` when the mock was called several times and the wrong call matched
- Mocks keeping calls from earlier tests because nothing cleared them (sheet 12)
- Over-mocking: tests that only check mocks talk to each other

✓ TIPS & BEST PRACTICES

- Inject dependencies as parameters so a `vi.fn()` is all you need
- Name mocks with `mockName` when several appear in one test
- Use `…Once` chains to test retry and fallback paths

QUICK SUMMARY

- `vi.fn()` — recording fake
- `mockReturnValue` / `mockResolvedValue` / `mockImplementation`
- `toHaveBeenCalledWith` / `…Times`
- `.mock.calls` — raw history

Spies & Resetting Mocks

WATCH REAL METHODS, THEN CLEAN UP

`vi.spyOn(object, "method")` wraps an existing method: by default the real code still runs, but calls are recorded. Add `.mockReturnValue` or `.mockImplementation` to replace it. Mocks keep their history and behavior until you **clear**, **reset** or **restore** them, so set that up once in config instead of in every file.

NOTES

- Spy on getters / setters with `vi.spyOn(obj, "prop", "get")`
- Only `mockRestore` puts the original method back
- `restoreMocks: true` in config restores spies after every test
- Spy on `console.error` to assert or silence warnings

SPIES AND CLEANUP

```ts
import * as api from './api'

test('logs a warning for large discounts', () => {
  const warn = vi.spyOn(console, 'warn').mockImplementation(() => {})
  applyDiscount(cart, 0.9)
  expect(warn).toHaveBeenCalledWith(expect.stringContaining('large discount'))
})

test('uses the cached price', () => {
  const fetchPrice = vi.spyOn(api, 'fetchPrice')     // real fn still runs
  getPrice('A1'); getPrice('A1')
  expect(fetchPrice).toHaveBeenCalledTimes(1)
})

test('getter', () => {
  vi.spyOn(navigator, 'onLine', 'get').mockReturnValue(false)
  expect(isOffline()).toBe(true)
})

// vitest.config.ts
// test: { restoreMocks: true }     // restore spies after each test

afterEach(() => {
  vi.clearAllMocks()                 // clear call history only
})
```

CODE EXPLAINED

→ `mockImplementation(() => {})` silences `console.warn` while still recording it

→ Spying on an ESM namespace export works in Vitest for many cases; module mocking (sheet 13) is the reliable route

→ The getter form needs the third argument `"get"`

→ `restoreMocks: true` is the simplest way to avoid spies leaking between tests

JEST ↔ VITEST

⇆ Jest: `jest.spyOn`, `jest.clearAllMocks`, same config keys

⇆ Difference: in Vitest 3+, `mockReset` restores the original implementation; in Jest it leaves an empty function

⇆ Vitest's `restoreAllMocks` only touches spies made with `vi.spyOn`

TYPESCRIPT

TS Spies are typed from the object: `vi.spyOn(api, "fetchPrice")` knows the signature

TS A wrong method name is a type error, not a run-time surprise

PREFER / AVOID

⇄ manual cleanup in every file → `restoreMocks: true`

⇄ replacing `console.error = vi.fn()` → `vi.spyOn(console, "error")`

⇄ spying on internals to test implementation → test outputs

CLEAR VS RESET VS RESTORE

METHOD	CALLS	BEHAVIOR	ORIGINAL
`mockClear()`	Cleared	Kept	Still mocked
`mockReset()`	Cleared	Back to original impl (Vitest) / empty (Jest)	Still mocked
`mockRestore()`	Cleared	Original	Put back (spies)

CONFIG & GLOBALS

ITEM	EFFECT
`clearMocks: true`	mockClear all before each test
`mockReset: true`	mockReset all before each test
`restoreMocks: true`	mockRestore spies before each test
`vi.clearAllMocks()`	Clear now
`vi.restoreAllMocks()`	Restore spies now

⚠ COMMON MISTAKES

- Expecting `mockClear` to remove a `mockReturnValue`
- Spying on `console.error` without restoring it, hiding errors in later tests
- Assuming Jest and Vitest `mockReset` behave the same during a migration

✓ TIPS & BEST PRACTICES

- Set `restoreMocks: true` in every project
- Fail tests on unexpected `console.error` with a spy in a setup file
- Spy, don't replace, when you only need to count calls

QUICK SUMMARY

- `vi.spyOn(obj, "m")` — record a real method
- clear → history · reset → behavior · restore → original
- `restoreMocks: true` — clean up automatically
- getter spies — third arg `"get"`

Sheet 13

Module Mocking

vi.mock(path, factory) · hoisting · importOriginal · partial mocks · __mocks__ folders · vi.doMock · vi.hoisted

REPLACE A WHOLE IMPORT

`vi.mock(path, factory)` replaces a module for every import in the test file. Vitest **hoists** `vi.mock` calls to the top of the file, before imports, so the mocked version is what your code gets. Return the fake exports from the factory; use **`importOriginal`** for a partial mock that keeps the real exports you don't override. Without a factory, Vitest automocks every export or uses a `__mocks__` file.

NOTES

- The path is resolved like an import from the test file
- Variables used in the factory must come from `vi.hoisted`
- Default exports need a `default` key in the factory
- `vi.doMock` is not hoisted; it affects later dynamic imports only

UNDER THE HOOD

- Vitest rewrites the file so `vi.mock` runs before any `import` statement
- That's why a factory can't use variables declared normally in the file: they don't exist yet
- Mocks apply per test file; each file has its own module graph

MOCKING IMPORTS

```ts
import { vi, test, expect } from 'vitest'
import { checkout } from './checkout'
import { charge } from './payments'

vi.mock('./payments', () => ({
  charge: vi.fn().mockResolvedValue({ id: 'ch_1' }),
}))

// partial mock: keep everything real except fetchRate
vi.mock('./rates', async (importOriginal) => ({
  ...(await importOriginal<typeof import('./rates')>()),
  fetchRate: vi.fn(() => 1.1),
}))

// variables for a factory must be hoisted too
const { sendMail } = vi.hoisted(() => ({ sendMail: vi.fn() }))
vi.mock('./mailer', () => ({ default: { send: sendMail } }))

test('checkout charges and emails', async () => {
  await checkout(cart)
  expect(charge).toHaveBeenCalledWith(cart.total)
  expect(sendMail).toHaveBeenCalledOnce()
})

vi.mock('node:fs')                  // automock, or __mocks__/fs.ts at the root
```

CODE EXPLAINED

- → `charge` is imported normally; because of hoisting it's the mocked version
- → `importOriginal` spreads the real module, then overrides one export
- → `vi.hoisted` makes `sendMail` available when the hoisted factory runs
- → A `default` key mocks a default export

JEST ↔ VITEST

- ⇆ Jest: `jest.mock(path, factory)` (hoisted by babel-jest), `jest.requireActual` for partials
- ⇆ Jest factories may only reference variables named `mock…`
- ⇆ Jest's ESM module mocking needs `jest.unstable_mockModule` + dynamic import

TYPESCRIPT

- TS Type the partial with `importOriginal<typeof import("./rates")>()`
- TS Use `vi.mocked(charge)` to get a typed mock of an imported function
- TS `vi.mock(import("./payments"), …)` gives the factory typed exports

PREFER / AVOID

- ⇄ mocking your own internal modules everywhere → inject dependencies or mock at the network edge (MSW)
- ⇄ full factory copying every export → `importOriginal` partial mock
- ⇄ variables declared above `vi.mock` → `vi.hoisted`

MODULE MOCK API

CALL	DOES
`vi.mock(path, factory)`	Replace (hoisted)
`vi.mock(path)`	Automock or __mocks__
`importOriginal()`	Real module inside a factory
`vi.importActual(path)`	Real module anywhere
`vi.doMock(path, f)`	Not hoisted; next dynamic import
`vi.unmock(path)`	Stop mocking
`vi.hoisted(fn)`	Values usable in factories

__MOCKS__ FOLDERS

MOCKING	FILE
`A local module ./api.ts`	src/__mocks__/api.ts next to it
`A package (axios)`	__mocks__/axios.ts at the project root
`A Node built-in (fs)`	__mocks__/fs.cjs at the root
`Activate it`	vi.mock(path) with no factory

⚠ COMMON MISTAKES

- Referencing a normal variable inside the factory ("Cannot access before initialization")
- Forgetting `default` when the module has a default export
- Mocking a module in one file and expecting it in another

✓ TIPS & BEST PRACTICES

- Prefer MSW (sheet 15) over mocking your HTTP client module
- Use partial mocks so new exports don't break tests
- Check `vi.mocked(fn).mock.calls` when an assertion on a mocked import fails

QUICK SUMMARY

- `vi.mock(path, factory)` — hoisted replacement
- `importOriginal` — partial mocks
- `vi.hoisted` — values for factories
- `__mocks__/` — reusable mocks

Fake Timers & Dates

`vi.useFakeTimers` · `advanceTimersByTime` · `runAllTimers` · `runOnlyPendingTimers` · `vi.setSystemTime` · `useRealTimers`

CONTROL TIME INSTEAD OF WAITING

Fake timers replace `setTimeout`, `setInterval`, `Date` and friends with versions you move forward by hand. A debounce of 300 ms takes zero real time to test: `vi.advanceTimersByTime(300)`. `vi.setSystemTime` pins "now" so date logic is predictable. Always switch back with `vi.useRealTimers()`.

NOTES

- Use the `…Async` versions when timers trigger promises
- `runAllTimers` loops forever on a self-rescheduling interval; use `runOnlyPendingTimers`
- `setSystemTime` also accepts `Temporal.Instant` (Vitest 5.1)
- Fake timers + `.concurrent` tests don't mix

UNDER THE HOOD

- Vitest uses @sinonjs/fake-timers to swap the global timer functions and `Date`
- `toFake` chooses which APIs are replaced; `nextTick` and `queueMicrotask` are left real by default
- Libraries that saved a reference to the real `setTimeout` before faking aren't affected

DEBOUNCE, INTERVAL, DATES

```ts
import { vi, beforeEach, afterEach, test, expect } from 'vitest'

beforeEach(() => { vi.useFakeTimers() })
afterEach(() => { vi.useRealTimers() })

test('debounced search fires once', () => {
  const search = vi.fn()
  const onType = debounce(search, 300)
  onType('a'); onType('ap'); onType('app')
  vi.advanceTimersByTime(299)
  expect(search).not.toHaveBeenCalled()
  vi.advanceTimersByTime(1)
  expect(search).toHaveBeenCalledExactlyOnceWith('app')
})

test('polls every 5 s', async () => {
  const tick = vi.fn().mockResolvedValue(undefined)
  startPolling(tick, 5000)
  await vi.advanceTimersByTimeAsync(15_000)
  expect(tick).toHaveBeenCalledTimes(3)
})

test('trial expires after 14 days', () => {
  vi.setSystemTime(new Date('2026-11-01T00:00:00Z'))
  const trial = startTrial()
  vi.setSystemTime(new Date('2026-11-15T00:00:01Z'))
  expect(trial.expired()).toBe(true)
})
```

CODE EXPLAINED

- → Moving 299 ms then 1 ms shows the debounce fires exactly at 300
- → `advanceTimersByTimeAsync` also flushes promises between timers
- → `setSystemTime` changes what `new Date()` and `Date.now()` return
- → Restoring real timers in `afterEach` keeps other tests safe

JEST ↔ VITEST

- ⇆ Jest: `jest.useFakeTimers()` (modern timers are the default), `jest.advanceTimersByTime`, `jest.setSystemTime`
- ⇆ Same `…Async` helpers exist in Jest 29.5+

TYPESCRIPT

- TS Accept a clock in your code (`now: () => Date`) and pass a fake; no global faking needed
- TS `vi.setSystemTime` takes `Date | number | string`

PREFER / AVOID

- ⇄ `await sleep(300)` in tests → fake timers
- ⇄ faking timers in every file → only where time matters, restore in `afterEach`
- ⇄ `new Date()` buried in code → inject a clock

TIMER API

CALL	DOES
`vi.useFakeTimers(opts?)`	Start faking
`vi.advanceTimersByTime(ms)`	Move forward
`vi.advanceTimersToNextTimer()`	Jump to next timer
`vi.runOnlyPendingTimers()`	Run timers queued now
`vi.runAllTimers()`	Run until none left
`vi.setSystemTime(d)`	Set now
`vi.useRealTimers()`	Stop faking

WHAT GETS FAKED

API	DEFAULT
`setTimeout / setInterval`	Faked
`Date / Date.now`	Faked
`setImmediate`	Faked
`performance.now`	Faked
`process.nextTick / queueMicrotask`	Real unless in toFake

⚠ COMMON MISTAKES

- Forgetting `useRealTimers`, so later tests hang waiting on real timeouts
- `runAllTimers` on an interval and getting an "aborting after 10000 timers" error
- Awaiting a promise that waits on a timer you never advance

✓ TIPS & BEST PRACTICES

- Use the `Async` variants whenever promises are involved
- Pin the system time in tests that format dates
- Prefer injecting a clock for business logic; keep fake timers for UI debounces and polling

QUICK SUMMARY

- `vi.useFakeTimers()` / `useRealTimers()`
- `advanceTimersByTime(ms)` — move time
- `…Async` — flush promises too
- `vi.setSystemTime` — pin now

Sheet 15

Mocking Network, Globals & Env

MSW `setupServer` · `http.get` handlers · `server.use` overrides · `vi.stubGlobal('fetch')` · `vi.stubEnv` · `unstubAll`

FAKE THE NETWORK, NOT YOUR CODE

MSW (Mock Service Worker) intercepts real HTTP requests made with `fetch`, axios or anything else and answers from **handlers** you write. Your code runs unchanged, which makes it the most realistic way to test API calls. For quick cases, `vi.stubGlobal` replaces a global like `fetch`, and `vi.stubEnv` sets `import.meta.env` / `process.env` values.

NOTES

- `npm i -D msw` (v2 API: `http`, `HttpResponse`)
- Start the server in a setup file so every test file gets it
- `onUnhandledRequest: "error"` fails tests that hit an unmocked URL
- `unstubGlobals` / `unstubEnvs` config options undo stubs after each test

MSW SETUP AND OVERRIDES

```ts
// src/test/server.ts
import { setupServer } from 'msw/node'
import { http, HttpResponse } from 'msw'

export const server = setupServer(
  http.get('/api/users/:id', ({ params }) =>
    HttpResponse.json({ id: Number(params.id), name: 'Ann' })),
  http.post('/api/orders', async ({ request }) =>
    HttpResponse.json({ id: 'o1', ...(await request.json()) }, { status: 201 })),
)

// vitest.setup.ts
beforeAll(() => server.listen({ onUnhandledRequest: 'error' }))
afterEach(() => server.resetHandlers())
afterAll(() => server.close())

// a test
test('shows an error when the API fails', async () => {
  server.use(http.get('/api/users/:id', () => new HttpResponse(null, { status: 500 })))
  await expect(loadUser(1)).rejects.toThrow('500')
})

test('reads the API URL', () => {
  vi.stubEnv('VITE_API_URL', 'http://test')
  vi.stubGlobal('fetch', vi.fn().mockResolvedValue(new Response('{}')))
  // …
})
```

CODE EXPLAINED

- → Handlers match method and path, with `:id` params like a router
- → `server.use` adds an override for one test; `resetHandlers` removes it afterwards
- → `onUnhandledRequest: "error"` catches calls you forgot to mock
- → `vi.stubEnv` works for `import.meta.env` in Vite projects

JEST ↔ VITEST

- ⇆ MSW is runner-agnostic: the same server works in Jest
- ⇆ Jest has no `stubGlobal` / `stubEnv`; assign `global.fetch = jest.fn()` and restore it yourself

TYPESCRIPT

- TS Type responses: `http.get<{ id: string }, never, User>(…)`
- TS Share response types with the app so handlers break when the API type changes

PREFER / AVOID

- ⇄ mocking `axios` / `fetch` modules → MSW handlers
- ⇄ one handler per test file → shared default handlers + `server.use` overrides
- ⇄ tests calling real staging APIs → MSW in unit tests, real APIs only in E2E

MSW LIFECYCLE

CALL	WHEN
`server.listen()`	beforeAll
`server.use(...handlers)`	Inside a test
`server.resetHandlers()`	afterEach
`server.close()`	afterAll

STUBS

CALL	DOES
`vi.stubGlobal(name, value)`	Replace a global
`vi.unstubAllGlobals()`	Undo
`vi.stubEnv(name, value)`	Set env / import.meta.env
`vi.unstubAllEnvs()`	Undo
`unstubGlobals / unstubEnvs: true`	Auto-undo in config

⚠ COMMON MISTAKES

- Forgetting `resetHandlers`, so one test's override leaks into the next
- Relative URLs in Node without a base URL configured
- Leaving `onUnhandledRequest` at "warn" and missing a real network call

✓ TIPS & BEST PRACTICES

- Keep default handlers next to API types; override per test only the error paths
- Reuse the same MSW handlers in Storybook and the browser
- Turn on `unstubEnvs: true` so env stubs never leak

QUICK SUMMARY

- MSW — intercept real requests
- `server.use` — per-test overrides
- `vi.stubGlobal` — replace a global
- `vi.stubEnv` — env values

Snapshot Testing

COMPARE AGAINST A STORED RESULT

A **snapshot** test serializes a value and compares it with a stored copy. The first run writes the snapshot; later runs fail if the output changes, until you review and update it with –u. **Inline snapshots** are written into the test file itself, and Vitest's **file snapshots** compare against any file (HTML, JSON). Snapshots are best for output that's tedious to assert by hand.

NOTES

- Snapshots live in `__snapshots__/<file>.snap`; commit them
- Review snapshot diffs in PRs like code
- Obsolete snapshots are reported; `-u` removes them
- In CI, missing snapshots fail instead of being written

SNAPSHOT KINDS

```
test('renders an invoice', () => {
  expect(renderInvoice(order)).toMatchSnapshot()
})

test('formats an address', () => {
  expect(formatAddress(addr)).toMatchInlineSnapshot(`
    "Bob Smith
    1 Main St
    Tripoli, IA 50676"
  `)                                    // filled in on first run
})

test('email HTML', async () => {
  await expect(renderEmail(order)).toMatchFileSnapshot('./__snapshots__/email.html')
})

test('dynamic fields', () => {
  expect(createOrder()).toMatchSnapshot({
    id: expect.any(String),             // property matchers
    createdAt: expect.any(Date),
  })
})

// $ npx vitest -u     update snapshots after reviewing the change
```

CODE EXPLAINED

- → `toMatchInlineSnapshot()` with no argument writes the value into the test source on the first run
- → File snapshots keep large output in its own file with proper syntax highlighting
- → Property matchers stop changing IDs and dates from breaking the snapshot
- → `u` in watch mode updates the failing snapshots

JEST ↔ VITEST

- ⇆ Jest has `toMatchSnapshot` and `toMatchInlineSnapshot` (needs Prettier 3 handling in Jest 30)
- ⇆ No file snapshots in Jest
- ⇆ Snapshot file headers differ (`// Vitest Snapshot v1` vs `// Jest Snapshot v1`)

TYPESCRIPT

- TS Snapshots serialize values, not types; use `expectTypeOf` for API type shapes
- TS Inline snapshots are template strings; avoid backticks in snapshotted text

PREFER / AVOID

- ⇄ snapshotting whole React trees → snapshot small, stable output or assert with Testing Library
- ⇄ running `-u` without reading the diff → review every change
- ⇄ huge `.snap` files nobody reads → inline or file snapshots, or real assertions

SNAPSHOT MATCHERS

MATCHER	STORES IN
`toMatchSnapshot()`	__snapshots__/*.snap
`toMatchInlineSnapshot()`	The test file
`toMatchFileSnapshot(path)`	Any file (Vitest)
`toThrowErrorMatchingSnapshot()`	Error message
`toMatchScreenshot()`	Image (browser mode, sheet 19)

GOOD VS BAD USES

GOOD	BAD
`Serializer / formatter output`	Large component trees
`Error messages`	Output with timestamps
`CLI help text`	Random IDs (without matchers)
`Generated SQL / config`	Anything nobody will review

⚠ COMMON MISTAKES

- Updating snapshots to make CI green without checking the change was intended
- Snapshotting values with dates or random IDs
- Snapshots so large the diff is unreadable

✓ TIPS & BEST PRACTICES

- Prefer inline snapshots for small values; they're reviewed in the test itself
- Use property matchers or serializers to normalize volatile fields
- Delete obsolete snapshots regularly with `-u`

QUICK SUMMARY

- `toMatchSnapshot` — stored file
- `toMatchInlineSnapshot` — in the test
- `toMatchFileSnapshot` — any file (Vitest)
- `-u` — update after review

DOM Environments

A FAKE BROWSER IN NODE

Tests run in Node, which has no `document` or `window`. A **DOM environment** adds them: **jsdom** (most complete, slower) or **happy-dom** (faster, a little less complete). Set it globally in config or per file with a comment, so pure logic tests stay on the faster `node` environment. Neither does layout or real rendering; for that, use browser mode (sheet 19) or Playwright.

NOTES

- Install the one you use: `npm i -D jsdom` or `happy-dom`
- Per-file comment must be the first comment in the file
- Missing APIs (`matchMedia`, `IntersectionObserver`, `ResizeObserver`) need small stubs
- `getBoundingClientRect()` returns zeros: there's no layout

UNDER THE HOOD

- jsdom and happy-dom are JavaScript re-implementations of the DOM standards, not browsers
- They parse HTML and run events, but compute no CSS layout and paint nothing
- Anything that depends on sizes, scrolling or visibility needs a real browser

CHOOSING AN ENVIRONMENT

```ts
// vitest.config.ts — default for all tests
test: { environment: 'node' }

// Button.test.tsx — this file only
// @vitest-environment jsdom
import { test, expect } from 'vitest'

test('adds a class on click', () => {
  document.body.innerHTML = '<button id="b">Save</button>'
  const btn = document.getElementById('b')!
  attachToggle(btn)
  btn.click()
  expect(btn.classList.contains('active')).toBe(true)
})

// vitest.setup.ts — stub what jsdom lacks
Object.defineProperty(window, 'matchMedia', {
  value: (query: string) => ({
    matches: false, media: query,
    addEventListener() {}, removeEventListener() {},
  }),
})

// projects: node for logic, jsdom for components (sheet 21)
```

CODE EXPLAINED

→ `node` is the default and the fastest; only DOM tests need jsdom

→ The `@vitest-environment` comment overrides config for one file

→ `matchMedia` is missing in jsdom; many UI libraries call it on import

→ Projects (sheet 21) can assign environments by folder instead of comments

JEST ↔ VITEST

⇆ Jest: `testEnvironment: "jsdom"` needs the `jest-environment-jsdom` package (Jest 28+); Jest 30 ships jsdom 26

⇆ Per file: `/** @jest-environment jsdom */`

TYPESCRIPT

TS Add `"lib": ["dom"]` and DOM types in the test tsconfig

TS Non-null assertions (`!`) are common for `getElementById`; Testing Library queries avoid them

PREFER / AVOID

⇄ jsdom for every test file → `node` default + jsdom per file or project

⇄ testing layout / scroll in jsdom → browser mode or Playwright

⇄ hand-written `innerHTML` + `querySelector` → Testing Library (sheet 18)

ENVIRONMENTS

ENVIRONMENT	SPEED	USE FOR
`node`	Fastest	Logic, APIs, utilities
`happy-dom`	Fast	Most component tests
`jsdom`	Slower	Best DOM compatibility
`edge-runtime`	Fast	Edge / worker code
`Browser mode`	Slowest	Real layout and events

NOT IN JSDOM / HAPPY-DOM

API	WORKAROUND
`Layout (sizes, positions)`	Browser mode
`matchMedia`	Stub in setup
`IntersectionObserver / ResizeObserver`	Stub or polyfill
`Canvas drawing`	canvas package or browser mode
`Navigation (window.location changes)`	Mock the router

⚠ COMMON MISTAKES

- Running every test in jsdom and wondering why the suite is slow
- Asserting element sizes or visibility based on CSS in jsdom
- Placing the environment comment after imports, where it's ignored

✓ TIPS & BEST PRACTICES

- Try happy-dom first; switch to jsdom when something is missing
- Keep a shared setup file with the standard stubs
- Move layout-dependent tests to browser mode

QUICK SUMMARY

- jsdom / happy-dom — DOM in Node
- `// @vitest-environment jsdom` — per file
- no layout — sizes are zero
- stub missing APIs in setup

Testing Library

render · screen · getByRole / findBy* / queryBy* · userEvent · @testing-library/jest-dom matchers · cleanup

TEST COMPONENTS LIKE A USER

Testing Library renders a component and lets you find elements the way a user would: by **role**, label and text, not by CSS class. **user-event** simulates real typing and clicking, and **jest-dom** adds matchers like `toBeInTheDocument` and `toBeDisabled`. There are versions for React, Vue, Svelte and more; this sheet uses React. The React set covers the components themselves.

NOTES

- `npm i -D @testing-library/react @testing-library/user-event @testing-library/jest-dom`
- `getBy*` throws if missing, `queryBy*` returns null, `findBy*` waits (async)
- Import `@testing-library/jest-dom/vitest` in the setup file
- Cleanup is automatic when `globals: true`; otherwise call `cleanup` in `afterEach`

A COMPONENT TEST

```
import { render, screen } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { LoginForm } from './LoginForm'

test('submits email and password', async () => {
  const user = userEvent.setup()
  const onSubmit = vi.fn()
  render(<LoginForm onSubmit={onSubmit} />)

  await user.type(screen.getByLabelText(/email/i), 'ann@example.com')
  await user.type(screen.getByLabelText(/password/i), 'hunter2')
  await user.click(screen.getByRole('button', { name: /sign in/i }))

  expect(onSubmit).toHaveBeenCalledWith({ email: 'ann@example.com', password: 'hunter2' })
})

test('shows an error from the server', async () => {
  server.use(http.post('/api/login', () => new HttpResponse(null, { status: 401 })))
  render(<LoginForm onSubmit={login} />)
  await userEvent.click(screen.getByRole('button', { name: /sign in/i }))
  expect(await screen.findByRole('alert')).toHaveTextContent(/wrong password/i)
  expect(screen.queryByText(/welcome/i)).not.toBeInTheDocument()
})
```

CODE EXPLAINED

- → `getByLabelText` finds the input a `<label>` points to, which also checks accessibility
- → `userEvent.setup()` then `await user.type(...)` fires the full sequence of key events
- → `findByRole` waits for the alert to appear after the async request
- → `queryBy…` + `not.toBeInTheDocument()` asserts something is absent

JEST ↔ VITEST

- ⇆ Same libraries in Jest; import `@testing-library/jest-dom` (no `/vitest`) in `setupFilesAfterEnv`
- ⇆ Browser mode uses `vitest-browser-react` instead (sheet 19)

TYPESCRIPT

- TS Add `"types": ["@testing-library/jest-dom"]` so matchers are typed
- TS Queries return `HTMLElement`; narrow with `toBeInstanceOf(HTMLInputElement)` when needed

PREFER / AVOID

- ⇄ `getByTestId` everywhere → `getByRole` / `getByLabelText`
- ⇄ `fireEvent.change` → `user.type`
- ⇄ `waitFor(() => getBy…)` → `findBy…`

QUERY PRIORITY

QUERY	USE FOR
`getByRole(role, { name })`	Buttons, links, headings, inputs (first choice)
`getByLabelText`	Form fields
`getByPlaceholderText`	Inputs without labels
`getByText`	Non-interactive text
`getByDisplayValue`	Current input value
`getByAltText / getByTitle`	Images, titles
`getByTestId`	Last resort

JEST-DOM MATCHERS

MATCHER	CHECKS
`toBeInTheDocument()`	In the DOM
`toBeVisible()`	Not hidden
`toBeDisabled() / toBeEnabled()`	Disabled state
`toHaveTextContent(t)`	Text
`toHaveValue(v)`	Input value
`toHaveAttribute(a, v?)`	Attribute
`toHaveAccessibleName(n)`	Accessible name

⚠ COMMON MISTAKES

- Querying by class names or DOM structure that changes with styling
- Using `getBy…` for something that appears later; use `findBy…`
- Forgetting `await` on `user` events

✓ TIPS & BEST PRACTICES

- If you can't find an element by role, the component may have an accessibility problem
- Use `screen.debug()` to print the current DOM
- Use Testing Playground (`screen.logTestingPlaygroundURL()`) to find the best query

QUICK SUMMARY

- `render` + `screen` — mount and query
- `getByRole` — first-choice query
- `userEvent` — real interactions
- jest-dom — DOM matchers

Vitest Browser Mode

`browser.enabled` · Playwright provider · `page` locators · `expect.element` · visual regression · when to use Playwright Test instead

COMPONENT TESTS IN A REAL BROWSER

Browser mode runs Vitest tests inside a real browser (Chromium, Firefox, WebKit) driven by Playwright, so layout, CSS, focus and real events work. Tests use the same `describe` / `it` API, with `page` locators and `expect.element`, which retries until the element matches. It sits between jsdom unit tests and full end-to-end tests in Playwright.

NOTES

- Install: `npm i -D @vitest/browser-playwright` and run `npx playwright install chromium`
- Render components with `vitest-browser-react` (or `-vue`, `-svelte`)
- `headless: true` for CI
- Visual regression: `toMatchScreenshot()` compares images

UNDER THE HOOD

- Vitest serves test files through Vite and opens them in a browser controlled by the provider (Playwright)
- The test code runs in the page; `page` commands go back to the Node side over a connection
- Real layout and events come from the browser, so no DOM stubs are needed

CONFIG AND A TEST

```ts
// vitest.config.ts
import { defineConfig } from 'vitest/config'
import { playwright } from '@vitest/browser-playwright'

export default defineConfig({
  test: {
    browser: {
      enabled: true,
      provider: playwright(),
      headless: true,
      instances: [{ browser: 'chromium' }],
    },
  },
})

// Menu.browser.test.tsx
import { render } from 'vitest-browser-react'
import { page } from 'vitest/browser'

test('opens the menu and focuses the first item', async () => {
  render(<Menu items={['New', 'Open']} />)
  await page.getByRole('button', { name: 'Menu' }).click()
  await expect.element(page.getByRole('menuitem', { name: 'New' })).toHaveFocus()
  await expect(page.getByRole('menu')).toMatchScreenshot()
})
```

CODE EXPLAINED

- → `instances` lists browsers; add `firefox` or `webkit` to test more than one
- → `page` locators work like Playwright's (`getByRole`, `getByText`)
- → `expect.element` retries, so no manual waits are needed
- → Screenshot baselines are stored per browser and OS

JEST ↔ VITEST

- ⇆ Jest has no browser mode; it runs only in jsdom / Node
- ⇆ Closest Jest option is jest-puppeteer or moving the test to Playwright

TYPESCRIPT

- TS Add `"types": ["vitest/browser"]` for typed `page` APIs
- TS TSX component tests run directly through Vite

PREFER / AVOID

- ⇄ jsdom stubs for layout-heavy components → browser mode
- ⇄ full user journeys in browser mode → Playwright Test (Playwright set)
- ⇄ screenshots of whole pages → small, stable component screenshots

WHICH KIND OF TEST?

NEED	USE
Pure logic	Vitest, node env
Component markup and events	Vitest + jsdom / happy-dom
Layout, focus, CSS, real events	Vitest browser mode
Full app across pages, auth, network	Playwright Test

BROWSER OPTIONS

OPTION	SETS
`enabled`	Turn browser mode on
`provider`	playwright() / webdriverio()
`instances`	Browsers to run
`headless`	No window
`viewport`	Page size
`screenshotFailures`	Save a screenshot on failure

⚠ COMMON MISTAKES

- Running browser mode in CI without installing browsers (`npx playwright install --with-deps`)
- Using browser mode for everything and making the suite slow
- Screenshot baselines made on macOS and compared on Linux CI

✓ TIPS & BEST PRACTICES

- Run browser-mode tests as a separate project (sheet 21)
- Generate screenshot baselines in the same Docker image CI uses
- Reuse Playwright locator skills from the Playwright set

QUICK SUMMARY

- browser mode — real browser via Playwright
- `page` locators + `expect.element`
- `toMatchScreenshot` — visual checks
- full journeys → Playwright Test

Coverage

`--coverage` · `provider: 'v8'` vs `'istanbul'` · `thresholds` · `include` / `exclude` · reporters (`text` · `html` · `lcov`) · ignore comments

WHICH CODE THE TESTS RAN

`vitest run --coverage` reports which lines, branches and functions ran. The **v8** provider (default) reads V8's built-in counters and is fast; **istanbul** instruments code first and works in any JavaScript engine. Set `include` so untested files count, and `thresholds` to fail CI when coverage drops.

NOTES

- Install the provider: `npm i -D @vitest/coverage-v8`
- Without `include`, files no test imports are left out of the report
- `html` report: open `coverage/index.html`
- Coverage shows what ran, not whether the assertions were good

UNDER THE HOOD

- v8 coverage uses counters the engine keeps anyway, then maps them back through source maps
- istanbul rewrites code with counters before running it
- Vitest's v8 provider remaps results to match istanbul accuracy, but small differences remain

COVERAGE CONFIG

```ts
// vitest.config.ts
test: {
  coverage: {
    provider: 'v8',                    // or 'istanbul'
    include: ['src/**/*.{ts,tsx}'],
    exclude: ['src/**/*.stories.tsx', 'src/test/**', '**/*.d.ts'],
    reporter: ['text', 'html', 'lcov'],
    thresholds: {
      lines: 80,
      branches: 75,
      functions: 80,
      'src/billing/**': { lines: 95 },  // stricter for one folder
    },
  },
}

// $ npx vitest run --coverage
// File       | % Stmts | % Branch | % Funcs | % Lines | Uncovered
// cart.ts    |   92.3  |   83.3   |   100   |   92.3  | 41-43

/* v8 ignore next */
if (import.meta.env.DEV) debugPanel()
```

CODE EXPLAINED

- → `include` makes untested files show up at 0% instead of disappearing
- → Glob keys in `thresholds` set stricter rules for critical folders
- → `lcov` feeds Codecov, Coveralls and editor gutter extensions
- → `/* v8 ignore next */` (or `/* istanbul ignore next */`) skips a line or block

JEST ↔ VITEST

- ⇆ Jest: `jest --coverage`, `coverageProvider: "v8" | "babel"`, `collectCoverageFrom`, `coverageThreshold`
- ⇆ Jest uses istanbul (babel) by default

TYPESCRIPT

- TS Exclude `*.d.ts` and type-only files; they have no runtime code
- TS Coverage maps to your `.ts` source through source maps

PREFER / AVOID

- ⇄ chasing 100% → meaningful tests on risky code
- ⇄ ignore comments everywhere → `exclude` patterns
- ⇄ no `include` → `include: ["src/**"]` so gaps show

PROVIDERS

PROVIDER	PACKAGE	NOTES
`v8 (default)`	@vitest/coverage-v8	Fast, Node / Chromium
`istanbul`	@vitest/coverage-istanbul	Any runtime, slower

COVERAGE OPTIONS

OPTION	SETS
`include / exclude`	Files counted
`reporter`	text · html · lcov · json · cobertura
`thresholds`	Minimums (global or per glob)
`reportsDirectory`	Output folder
`reportOnFailure`	Report even if tests fail
`thresholds.autoUpdate`	Raise thresholds as coverage grows

⚠ COMMON MISTAKES

- Forgetting `include`, so coverage looks high because untested files aren't counted
- Turning on coverage in watch mode and slowing every rerun
- Treating the number as proof of quality

✓ TIPS & BEST PRACTICES

- Run coverage only in CI or on demand
- Use per-folder thresholds for critical code
- Read the uncovered lines in the HTML report, not just the percentages

QUICK SUMMARY

- `--coverage` — measure
- v8 (fast) vs istanbul (portable)
- `include` — count untested files
- `thresholds` — fail below a floor

Projects & Performance

ONE REPO, SEVERAL TEST SETUPS, FAST RUNS

Projects run different test setups in one Vitest command: unit tests in `node`, component tests in jsdom, browser tests in Chromium, or one project per package in a monorepo. Speed comes from parallel **workers** (`pool`), skipping per-file isolation when tests are safe, **sharding** across CI machines, and running only tests affected by a change.

NOTES

- `projects` replaced the old `vitest.workspace` file
- Default pool is `forks` (child processes); `threads` can be faster
- `isolate: false` reuses module state between files: only for pure tests
- Slow tests are marked in the default reporter (over `slowTestThreshold`)

UNDER THE HOOD

- Each worker runs test files one at a time; files spread across workers
- `forks` uses child processes (most compatible); `threads` uses worker threads (less overhead)
- `--shard=1/3` splits the file list into thirds; each CI job runs one

PROJECTS AND SPEED SETTINGS

```ts
// vitest.config.ts
export default defineConfig({
  test: {
    projects: [
      { extends: true, test: { name: 'unit', environment: 'node',
          include: ['src/**/*.test.ts'] } },
      { extends: true, test: { name: 'dom', environment: 'happy-dom',
          include: ['src/**/*.test.tsx'] } },
      'packages/*',                    // each package's own config
    ],
    pool: 'threads',
    maxWorkers: 4,
    slowTestThreshold: 300,
  },
})

// $ npx vitest --project unit
// CI: shard across 3 jobs, then merge
// $ npx vitest run --shard=1/3 --reporter=blob
// $ npx vitest run --shard=2/3 --reporter=blob
// $ npx vitest --merge-reports --reporter=junit
```

CODE EXPLAINED

- → `extends: true` makes a project inherit the root config (plugins, aliases)
- → A glob like `packages/*` picks up each package's own Vitest config
- → `--reporter=blob` saves partial results that `--merge-reports` combines
- → `--project` runs one project locally

JEST ↔ VITEST

- ⇆ Jest: `projects: [...]` in `jest.config` with separate configs per project
- ⇆ Jest: `--shard=1/3`, `--maxWorkers`; workers are child processes
- ⇆ Jest has no `isolate: false` equivalent

TYPESCRIPT

- TS Give type tests their own project with `typecheck: { enabled: true }`
- TS Monorepo packages can each extend a shared base config

PREFER / AVOID

- ⇄ one config forcing jsdom on every file → `node` and `dom` projects
- ⇄ one long CI job → `--shard` across jobs
- ⇄ `vitest.workspace.ts` → `test.projects`

POOL OPTIONS

OPTION	EFFECT
`pool: "forks"`	Child processes (default, most compatible)
`pool: "threads"`	Worker threads (faster, some native modules break)
`maxWorkers`	Parallel worker cap
`isolate: false`	Reuse modules across files (faster, riskier)
`fileParallelism: false`	One file at a time

CI SPEED-UPS

TECHNIQUE	HOW
`Shard`	--shard=i/n + blob reporter + --merge-reports
`Affected only`	--changed origin/main
`Cache deps`	Cache node_modules / npm cache
`Split slow tests`	Separate project, run less often

⚠ COMMON MISTAKES

- `isolate: false` with tests that change globals, causing order-dependent failures
- `threads` with native modules that aren't thread-safe
- Projects without `extends: true` that lose the root plugins and aliases

✓ TIPS & BEST PRACTICES

- Start with defaults; profile before tuning pools
- Use `--changed` in PR builds and a full run on `main`
- Put browser-mode tests in their own project

QUICK SUMMARY

- `projects` — several setups in one run
- `pool` — forks (default) / threads
- `--shard` + blob — split across CI jobs
- `isolate: false` — faster, only for pure tests

Reporters & CI

OUTPUT FOR PEOPLE AND FOR CI

A **reporter** formats results. `default` shows a summary and failures; `verbose` lists every test; `dot` is compact; `junit` and `json` write files for CI dashboards; `github-actions` turns failures into annotations on the PR's changed lines. When the `CI` environment variable is set, Vitest runs once, disables watch mode and fails on `.only`.

NOTES

- Combine reporters: `--reporter=default --reporter=junit`
- `outputFile` sets where file reporters write
- `--bail N` stops after N failures
- The GitHub set covers workflow syntax

CI SETUP

```
# vitest.config.ts
# test: {
#   reporters: process.env.CI ? ['default', 'junit', 'github-actions'] : ['default'],
#   outputFile: { junit: './reports/junit.xml' },
# }

# .github/workflows/test.yml
name: test
on: [push, pull_request]
jobs:
  vitest:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v6
        with: { node-version: 24, cache: npm }
      - run: npm ci
      - run: npx tsc --noEmit
      - run: npx vitest run --coverage
      - uses: actions/upload-artifact@v7
        if: always()
        with: { name: reports, path: reports/ }
```

CODE EXPLAINED

- → Choosing reporters by `CI` keeps local output short and CI output complete
- → `github-actions` annotations appear on the exact failing lines in the PR
- → `tsc --noEmit` type-checks, since Vitest only strips types
- → `if: always()` uploads reports when tests fail too

JEST ↔ VITEST

- ⇆ Jest: `--reporters=default --reporters=jest-junit` (separate package)
- ⇆ Jest: `--ci` makes missing snapshots fail; Vitest does that automatically under `CI`
- ⇆ GitHub annotations in Jest need a third-party reporter

TYPESCRIPT

- TS Run `tsc --noEmit` (or `vitest --typecheck`) as its own CI step
- TS Type errors don't fail `vitest run`

PREFER / AVOID

- ⇄ parsing console output in CI → `junit` / `json` reporters
- ⇄ only running `vitest run` → add a separate type-check step
- ⇄ `--bail 1` in CI → see all failures; bail locally

BUILT-IN REPORTERS

REPORTER	OUTPUT
`default`	Summary + failures
`verbose`	Every test
`dot`	One dot per test
`tree`	Nested describe tree
`junit`	JUnit XML file
`json`	JSON file
`github-actions`	PR annotations
`blob`	Partial results for merge

CI BEHAVIOR (CI=TRUE)

BEHAVIOR	EFFECT
`Watch mode`	Off: runs once
`.only`	Fails the run (allowOnly: false)
`Missing snapshots`	Fail instead of being written
`Colors / TTY`	Plain output

⚠ COMMON MISTAKES

- Forgetting `outputFile` and losing the JUnit file in stdout
- No type-check step, so type errors merge
- Uploading artifacts only on success

✓ TIPS & BEST PRACTICES

- Use `github-actions` reporter on GitHub for inline failure annotations
- Upload the HTML coverage report as an artifact
- Keep the CI command identical to a local `npm run test:ci` script

QUICK SUMMARY

- `--reporter` — output format
- `junit` / `json` + `outputFile` — CI files
- `github-actions` — PR annotations
- `CI=true` — run once, strict

Sheet 23

Jest Setup & Config

jest.config.js · testEnvironment · transform (babel-jest · ts-jest · @swc/jest) · moduleNameMapper · setupFilesAfterEnv · ESM support

CONFIGURING JEST 30

Jest reads `jest.config.(js|ts|mjs|json)` or a `"jest"` key in `package.json`. Because Jest doesn't use Vite, it needs a **transform** for TypeScript and JSX (`babel-jest`, `ts-jest` or `@swc/jest`), **`moduleNameMapper`** for path aliases and CSS imports, and **`testEnvironment`** for DOM tests. ESM support still requires an experimental Node flag.

NOTES

- Jest 30 needs Node 18+ and TypeScript 5.4+
- `npx jest --init` writes a starter config
- `setupFilesAfterEnv` runs after the test framework loads: put matchers there
- Jest 30 removed several old matcher aliases and `jest.genMockFromModule`

JEST.CONFIG.TS

```ts
import type { Config } from 'jest'

const config: Config = {
  testEnvironment: 'jsdom',                    // needs jest-environment-jsdom
  transform: {
    '^.+\\.(t|j)sx?$': ['@swc/jest'],          // or 'ts-jest' / 'babel-jest'
  },
  moduleNameMapper: {
    '^@/(.*)$': '<rootDir>/src/$1',            // match tsconfig paths
    '\\.(css|scss)$': 'identity-obj-proxy',
  },
  setupFilesAfterEnv: ['<rootDir>/jest.setup.ts'],
  testMatch: ['**/?(*.)+(spec|test).[jt]s?(x)'],
  clearMocks: true,
  collectCoverageFrom: ['src/**/*.{ts,tsx}', '!src/**/*.d.ts'],
}
export default config

// ESM without a transform:
// $ node --experimental-vm-modules node_modules/jest/bin/jest.js
```

CODE EXPLAINED

- → `transform` maps file patterns to transformers; SWC is the fastest option
- → `moduleNameMapper` must mirror `tsconfig` paths and the bundler's aliases
- → `identity-obj-proxy` turns CSS-module imports into objects tests can read
- → A TS config file needs `ts-node` or Jest's built-in TS config loading

JEST ↔ VITEST

- ⇆ Vitest gets transforms, aliases and CSS handling from `vite.config` for free
- ⇆ `setupFilesAfterEnv` → Vitest `setupFiles`
- ⇆ `testEnvironment` → `environment`

TYPESCRIPT

- TS `ts-jest` type-checks while testing (slower); `@swc/jest` and `babel-jest` only strip types
- TS Use `import type { Config } from "jest"` for a typed config

PREFER / AVOID

- ⇄ `ts-jest` everywhere → `@swc/jest` for speed + separate `tsc` check
- ⇄ hand-copied aliases → generate `moduleNameMapper` from tsconfig (`pathsToModuleNameMapper` in ts-jest)
- ⇄ new Vite projects on Jest → Vitest

KEY OPTIONS

OPTION	SETS
`testEnvironment`	node (default) · jsdom
`transform`	File → transformer
`transformIgnorePatterns`	node_modules to transform (sheet 25)
`moduleNameMapper`	Aliases, asset stubs
`setupFilesAfterEnv`	Matchers, global mocks
`testMatch / testRegex`	Test files
`roots`	Folders to search
`clearMocks / restoreMocks`	Mock cleanup

TRANSFORMERS

TRANSFORMER	NOTES
`babel-jest`	Default when a Babel config exists
`ts-jest`	Type-checks; slowest
`@swc/jest`	Fast Rust-based
`esbuild-jest`	Fast, less maintained

⚠ COMMON MISTAKES

- Aliases that work in the app but not in Jest because `moduleNameMapper` is missing
- Using removed Jest 30 aliases (`toBeCalled` , `toThrowError`)
- ESM-only dependencies failing with "Cannot use import statement outside a module" (sheet 25)

✓ TIPS & BEST PRACTICES

- Keep Jest config small; most complexity is aliases and transforms
- Use `npx jest --showConfig` to see the merged config
- Consider migrating to Vitest if the project already uses Vite (sheet 24)

QUICK SUMMARY

- `jest.config.ts` — Jest settings
- `transform` — TS / JSX support
- `moduleNameMapper` — aliases and assets
- `setupFilesAfterEnv` — matchers and mocks

Migrating Jest → Vitest

MOSTLY FIND-AND-REPLACE

Vitest's API was designed to match Jest, so most suites move with a few mechanical changes: swap `jest.` for `vi.`, turn on `globals` (or add imports), and translate the config. The real work is in the differences: module mock factories, `done` callbacks, `mockReset` behavior, and anything that relied on Jest's module registry.

NOTES

- Migrate one folder at a time; both runners can live side by side
- Codemods exist (e.g. `vitest-codemod`); review the diff
- Snapshots regenerate on first run (`-u`)
- Run both suites in CI until the switch is done

BEFORE AND AFTER

```
// Jest                                    // Vitest
jest.fn()                                  vi.fn()
jest.spyOn(obj, 'm')                       vi.spyOn(obj, 'm')
jest.mock('./api')                         vi.mock('./api')
jest.requireActual('./api')                await vi.importActual('./api')
jest.useFakeTimers()                       vi.useFakeTimers()
jest.setTimeout(10_000)                    vi.setConfig({ testTimeout: 10_000 })

// Factories: Vitest needs every export the code uses, incl. default
jest.mock('./mailer', () => ({ send: jest.fn() }))
vi.mock('./mailer', () => ({ default: { send: vi.fn() }, send: vi.fn() }))

// done callbacks → promises
it('emits', (done) => { bus.on('x', () => done()) })            // Jest
it('emits', () => new Promise<void>((resolve) => bus.on('x', resolve)))

// config
// jest.config: testEnvironment, setupFilesAfterEnv, moduleNameMapper
// vitest:      environment,     setupFiles,         resolve.alias (from vite.config)
```

CODE EXPLAINED

- → `globals: true` lets old tests run without adding imports
- → Jest auto-fills a default export from CommonJS modules; Vitest needs an explicit `default` key
- → `done` callbacks become a returned promise
- → `importActual` is async in Vitest because ESM imports are async

JEST ↔ VITEST

- ⇆ `mockReset`: Jest leaves an empty function; Vitest 3+ restores the original implementation
- ⇆ Jest's `jest.isolateModules` → `vi.resetModules()` + dynamic import
- ⇆ `@testing-library/jest-dom` → import `@testing-library/jest-dom/vitest`

TYPESCRIPT

- TS Swap `@types/jest` for `"types": ["vitest/globals"]` in tsconfig
- TS Remove `ts-jest` / `@swc/jest`; Vite handles TS

PREFER / AVOID

- ⇄ big-bang migration → folder by folder with both runners
- ⇄ blindly accepting codemod output → review mock factories by hand
- ⇄ keeping `@types/jest` → remove it to avoid conflicting globals

CONFIG MAPPING

JEST	VITEST
`testEnvironment`	test.environment
`setupFilesAfterEnv`	test.setupFiles
`moduleNameMapper`	resolve.alias
`testMatch`	test.include
`clearMocks / restoreMocks`	Same names under test
`coverageThreshold`	test.coverage.thresholds
`projects`	test.projects

BEHAVIOR DIFFERENCES

AREA	WATCH FOR
`Mock factories`	Need default + all used exports
`done callbacks`	Not supported
`mockReset`	Restores original in Vitest
`Globals`	Off unless globals: true
`Snapshot header`	Rewritten on update
`jest.* in node_modules mocks`	Rename __mocks__ helpers

⚠ COMMON MISTAKES

- Leaving `@types/jest` installed, causing type conflicts with Vitest globals
- Forgetting the `default` key in mock factories
- Expecting `vi.importActual` to be synchronous

✓ TIPS & BEST PRACTICES

- Turn on `globals: true` first, then remove it later by adding imports
- Run the old and new suites side by side for a week
- Delete Jest config, transformers and types once the switch is done

QUICK SUMMARY

- `jest.` → `vi.`
- `globals: true` — run old tests as-is
- factories need `default` + used exports
- `done` → promises

Sheet 25

Jest in React Native

preset: 'react-native' · @testing-library/react-native · mocking native modules · transformIgnorePatterns · timers & animations

JEST IS STILL THE RN DEFAULT

React Native projects use **Jest**: new apps ship with the `react-native` preset (or `jest-expo` for Expo). Components are tested with **React Native Testing Library**, which renders to a test renderer instead of a device. Native modules don't exist in Node, so they must be **mocked**, and RN packages published as untranspiled ESM must be added to **`transformIgnorePatterns`**. The React Native set covers the app side.

NOTES

- `npm i -D @testing-library/react-native`
- Expo: `preset: "jest-expo"` handles most mocks for you
- Many libraries ship Jest mocks or a `setUpTests` helper; check their docs
- Device behavior (gestures, performance) needs Detox or Maestro end-to-end tests

JEST.CONFIG.JS AND A TEST

```
// jest.config.js
module.exports = {
  preset: 'react-native',                          // or 'jest-expo'
  setupFilesAfterEnv: ['<rootDir>/jest.setup.js'],
  transformIgnorePatterns: [
    'node_modules/(?!((jest-)?react-native|@react-native(-community)?|@react-navigation)/)',
  ],
}

// jest.setup.js
jest.mock('@react-native-async-storage/async-storage', () =>
  require('@react-native-async-storage/async-storage/jest/async-storage-mock'))

// Counter.test.tsx
import { render, screen, userEvent } from '@testing-library/react-native'

test('increments', async () => {
  const user = userEvent.setup()
  render(<Counter />)
  await user.press(screen.getByRole('button', { name: 'Add' }))
  expect(screen.getByText('Count: 1')).toBeOnTheScreen()
})
```

CODE EXPLAINED

- → The negative lookahead lets Jest transform the listed RN packages inside `node_modules`
- → Library-provided mocks (AsyncStorage here) are the safest way to fake native modules
- → RNTL's `userEvent.press` simulates a full press sequence
- → `toBeOnTheScreen` is one of RNTL's built-in matchers

JEST ↔ VITEST

- ⇆ Vitest has no official RN support; stay on Jest for React Native
- ⇆ Shared logic packages (no RN imports) can use Vitest

TYPESCRIPT

- TS RNTL queries are typed; `getByRole("button")` returns a host element
- TS Use `jest-expo` presets per platform (`jest-expo/ios`) for platform files

PREFER / AVOID

- ⇄ snapshotting whole screens → RNTL queries and assertions
- ⇄ hand-written mocks for popular libraries → their shipped Jest mocks
- ⇄ testing animations frame by frame → fake timers and assert end state

COMMON MOCKS

MODULE	HOW
`AsyncStorage`	Package's jest/async-storage-mock
`Reanimated`	Library's test setup helper (see its docs)
`react-native-gesture-handler`	jestSetup file it ships
`Native module (your own)`	jest.mock("./NativeThing", () => ({ … }))
`Navigation`	Wrap in a test NavigationContainer

RNTL ESSENTIALS

API	DOES
`render(<C />)`	Render to test renderer
`screen.getByRole / getByText`	Query
`userEvent.press / type`	Interact
`fireEvent.press`	Lower-level event
`toBeOnTheScreen()`	Element rendered
`toHaveTextContent(t)`	Text check

⚠ COMMON MISTAKES

- "SyntaxError: Cannot use import statement" from an RN package missing in `transformIgnorePatterns`
- Testing native behavior in Jest instead of Detox / Maestro
- Using the deprecated react-test-renderer directly for new tests

✓ TIPS & BEST PRACTICES

- Start from the template's Jest config and add packages to the ignore pattern as errors appear
- Use `jest-expo` in Expo apps; it ships the common mocks
- Keep business logic in plain TS modules so it's easy to test

QUICK SUMMARY

- `preset: "react-native"` / `"jest-expo"`
- RNTL — render, query, press
- mock native modules
- `transformIgnorePatterns` — transform RN packages

Quick Reference — Matchers & Mocks

matchers by category · asymmetric matchers · `vi` / `jest` mock API · clear / reset / restore table

EQUALITY & TRUTH (07)

MATCHER	CHECKS
`toBe`	Object.is
`toEqual`	Deep
`toStrictEqual`	Deep + types
`toBeNull / toBeUndefined`	Exact
`toBeDefined`	Not undefined
`toBeTruthy / toBeFalsy`	Truthiness
`toBeNaN`	NaN

NUMBERS, STRINGS, COLLECTIONS (08)

MATCHER	CHECKS
`toBeCloseTo(n, d)`	Float
`toBeGreaterThan(n)`	> n
`toMatch(re)`	String
`toContain(x)`	Item / substring
`toContainEqual(x)`	Item by value
`toHaveLength(n)`	Length
`toHaveProperty(p, v)`	Path
`toMatchObject(o)`	Subset

ERRORS & ASYNC (10)

FORM	CHECKS
`expect(() => f()).toThrow(x)`	Sync throw
`await expect(p).resolves.X`	Resolved value
`await expect(p).rejects.X`	Rejection
`expect.assertions(n)`	Count
`expect.soft(x)`	Continue
`expect.poll(fn)`	Retry

ASYMMETRIC (09)

MATCHER	MATCHES
`expect.any(C)`	Instance / type
`expect.anything()`	Not nullish
`expect.objectContaining(o)`	Has props
`expect.arrayContaining(a)`	Has items
`expect.stringMatching(re)`	Regex
`expect.closeTo(n)`	Float

CALL MATCHERS (11)

MATCHER	CHECKS
`toHaveBeenCalled()`	Called
`toHaveBeenCalledTimes(n)`	Count
`toHaveBeenCalledWith(…)`	Some call
`toHaveBeenLastCalledWith(…)`	Last call
`toHaveBeenNthCalledWith(n, …)`	nth call
`toHaveReturnedWith(v)`	Return value

CLEAR / RESET / RESTORE (12)

METHOD	EFFECT
`mockClear`	Calls cleared
`mockReset`	Calls + impl (Vitest: back to original)
`mockRestore`	Original put back
`clearMocks: true`	Auto clear
`restoreMocks: true`	Auto restore

VI ↔ JEST (11–15)

VITEST	JEST
`vi.fn()`	jest.fn()
`vi.spyOn(o, m)`	jest.spyOn(o, m)
`vi.mock(p, f)`	jest.mock(p, f)
`vi.importActual(p)`	jest.requireActual(p)
`vi.hoisted(fn)`	—
`vi.useFakeTimers()`	jest.useFakeTimers()
`vi.setSystemTime(d)`	jest.setSystemTime(d)
`vi.stubEnv / stubGlobal`	—

MOCK CONFIG (11)

METHOD	EFFECT
`mockReturnValue(v)`	Return
`mockResolvedValue(v)`	Resolve
`mockRejectedValue(e)`	Reject
`mockImplementation(f)`	Run f
`…Once variants`	Next call only

Sheet 27

Quick Reference — CLI & Config

Vitest CLI flags · Jest CLI flags · config keys side by side · environments · coverage options

VITEST CLI (03, 21, 22)

COMMAND	DOES
`vitest`	Watch
`vitest run`	Once
`vitest <filter>`	Path filter
`-t "name"`	Name filter
`--last-failed`	Previous failures
`--changed [ref]`	Affected tests
`--project name`	One project
`--shard=i/n`	Split
`--coverage`	Coverage
`-u`	Update snapshots
`--ui`	Dashboard

JEST CLI (23)

COMMAND	DOES
`jest`	Run once
`--watch / --watchAll`	Watch
`-t "name"`	Name filter
`-o / --onlyChanged`	Changed files
`--findRelatedTests f`	Related tests
`--coverage`	Coverage
`-u`	Update snapshots
`--ci`	Strict snapshots
`--shard=i/n`	Split
`--showConfig`	Print config

CONFIG SIDE BY SIDE (02, 23)

VITEST (TEST.*)	JEST
`environment`	testEnvironment
`include`	testMatch
`setupFiles`	setupFilesAfterEnv
`globals`	always on
`resolve.alias (Vite)`	moduleNameMapper
`(Vite transforms)`	transform
`coverage.thresholds`	coverageThreshold
`projects`	projects

ENVIRONMENTS (17, 19)

VALUE	PACKAGE
`node`	Built in
`jsdom`	jsdom (Vitest) · jest-environment-jsdom
`happy-dom`	happy-dom (Vitest)
`edge-runtime`	@edge-runtime/vm
`browser mode`	@vitest/browser-playwright

COVERAGE (20)

OPTION	SETS
`provider`	v8 · istanbul
`include / exclude`	Files counted
`reporter`	text · html · lcov · json
`thresholds`	Minimums
`/* v8 ignore next */`	Skip code